

第42章 I2S—音频播放与录音输入

本章参考资料：《STM32F4xx 参考手册》、《STM32F4xx 规格书》、库帮助文档《stm32f4xx_dsp_stdperiph_lib_um.chm》及《I2S BUS》。

若对 I2S 通讯协议不了解，可先阅读《I2S BUS》文档的内容学习。

关于音频编解码器 WM8978，请参考其规格书《WM8978_v4.5》来了解。

42.1 I2S 简介

Inter-IC Sound Bus(I2S)是飞利浦半导体公司(现为恩智浦半导体公司)针对数字音频设备之间的音频数据传输而制定的一种总线标准。在飞利浦公司的 I2S 标准中，既规定了硬件接口规范，也规定了数字音频数据的格式。

42.1.1 数字音频技术

现实生活中的声音是通过一定介质传播的连续的波，它可以由周期和振幅两个重要指标描述。正常人可以听到的声音频率范围为 20Hz~20KHz。现实存在的声音是模拟量，这对声音保存和长距离传输造成很大的困难，一般的做法是把模拟量转成对应的数字量保存，在需要还原声音的地方再把数字量的转成模拟量输出，参考图 42-1。



图 42-1 音频转换过程

模拟量转成数字量过程，一般可以分为三个过程，分别为采样、量化、编码，参考图 42-2。用一个比源声音频率高的采样信号去量化源声音，记录每个采样点的值，最后如果把所有采样点数值连接起来与源声音曲线是互相吻合的，只是它不是连续的。在图中两条蓝色虚线距离就是采样信号的周期，即对应一个采样频率(F_s)，可以想象得到采样频率越高最后得到的结果就与源声音越吻合，但此时采样数据量越越大，一般使用 44.1KHz 采样频率即可得到高保真的声音。每条蓝色虚线长度决定着该时刻源声音的量化值，该量化值有另外一个概念与之挂钩，就是量化位数。量化位数表示每个采样点用多少位表示数据范围，常用有 16bit、24bit 或 32bit，位数越高最后还原得到的音质越好，数据量也会越大。

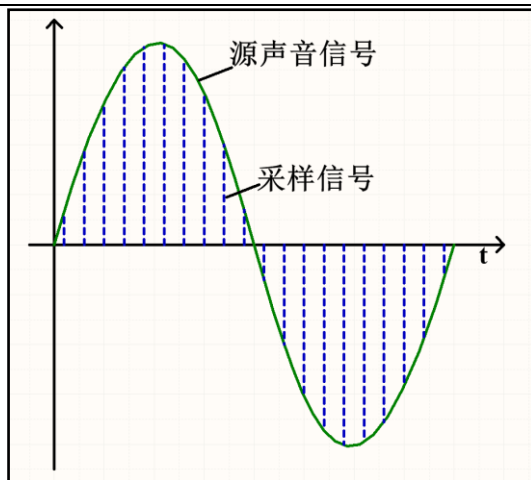


图 42-2 声音数字化过程

WM8978 是一个低功耗、高质量的立体声多媒体数字信号编译码器，集成 DAC 和 ADC，可以实现声音信号量化成数字量输出，也可以实现数字量音频数据转换为模拟量声音驱动扬声器。这样使用 WM8978 芯片解决了声音与数字量音频数据转换问题，并且通过配置 WM8978 芯片相关寄存器可以控制转换过程的参数，比如采样频率，量化位数，增益、滤波等等。

WM8978 芯片是一个音频编译码器，但本身没有保存音频数据功能，它只能接收其它设备传输过来的音频数据进行转换输出到扬声器，或者把采样到的音频数据输出到其它具有存储功能的设备保存下来。该芯片与其他设备进行音频数据传输接口就是 I2S 协议的音频接口。

42.1.2 I2S 总线接口

I2S 总线接口有 3 个主要信号，但只能实现数据半双工传输，后来为实现全双工传输有些设备增加了扩展数据引脚。STM32f4xx 系列控制器支持扩展的 I2S 总线接口。

- (1) SD(Serial Data): 串行数据线，用于发送或接收两个时分复用的数据通道上的数据(仅半双工模式)，如果是全双工模式，该信号仅用于发送数据。
- (2) WS(Word Select): 字段选择线，也称帧时钟(LRC)线，表明当前传输数据的声道，不同标准有不同的定义。WS 线的频率等于采样频率(F_s)。
- (3) CK(Serial Clock): 串行时钟线，也称位时钟(BCLK)，数字音频的每一位数据都对应有一个 CK 脉冲，它的频率为： $2 \times \text{采样频率} \times \text{量化位数}$ ，2 代表左右两个通道数据。
- (4) ext_SD(extend Serial Data): 扩展串行数据线，用于全双工传输的数据接收。

另外，有时为使系统间更好地同步，还要传输一个主时钟(MCK)，STM32F4xx 系列控制器固定输出为 $256 \times F_s$ 。

42.1.3 音频数据传输协议标准

随着技术的发展，在统一的 I2S 硬件接口下，出现了多种不同的数据格式，可分为左对齐(MSB)标准、右对齐(LSB)标准、I2S Philips 标准。另外，STM32F4xx 系列控制器还支持

持 PCM(脉冲编码调)音频传输协议。下面以 STM32F4xx 系列控制器资源解释这四个传输协议。

STM32f4xx 系列控制器 I2S 的数据寄存器只有 16bit，并且左右声道数据一般是紧邻传输，为正确得到左右两个声道数据，需要软件控制数据对应通道数据写入或读取。另外，音频数据的量化位数可能不同，控制器支持 16bit、24bit 和 32bit 三种数据长度，因为数据寄存器是 16bit 的，所以对于 24bit 和 32bit 数据长度需要发送两个。为此，可以产生四种数据和帧格式组合：

- ❑ 将 16 位数据封装在 16 位帧中
- ❑ 将 16 位数据封装在 32 位帧中
- ❑ 将 24 位数据封装在 32 位帧中
- ❑ 将 32 位数据封装在 32 位帧中

当使用 32 位数据包中的 16 位数据时，前 16 位(MSB)为有效位，16 位 LSB 被强制清零，无需任何软件操作或 DMA 请求（只需一个读/写操作）。如果程序使用 DMA 传输(一般都会用)，则 24 位和 32 位数据帧需要对数据寄存器执行两次 DMA 操作。24 位的数据帧，硬件会将 8 位非有效位扩展到带有 0 位的 32 位。对于所有数据格式和通信标准而言，始终会先发送最高有效位(MSB 优先)。

1. I2S Philips 标准

使用 WS 信号来指示当前正在发送的数据所属的通道，为 0 时表示左通道数据。该信号从当前通道数据的第一个位(MSB)之前的一个时钟开始有效。发送方在时钟信号(CK)的下降沿改变数据，接收方在上升沿读取数据。WS 信号也在 SCK 的下降沿变化。参考图 42-3，为 24bit 数据封装在 32bit 帧传输波形。正如之前所说，WS 线频率对于采样频率 F_s ，一个 WS 线周期包括发送左声道和右声道数据，在图中实际需要 64 个 CK 周期来完成一次传输。

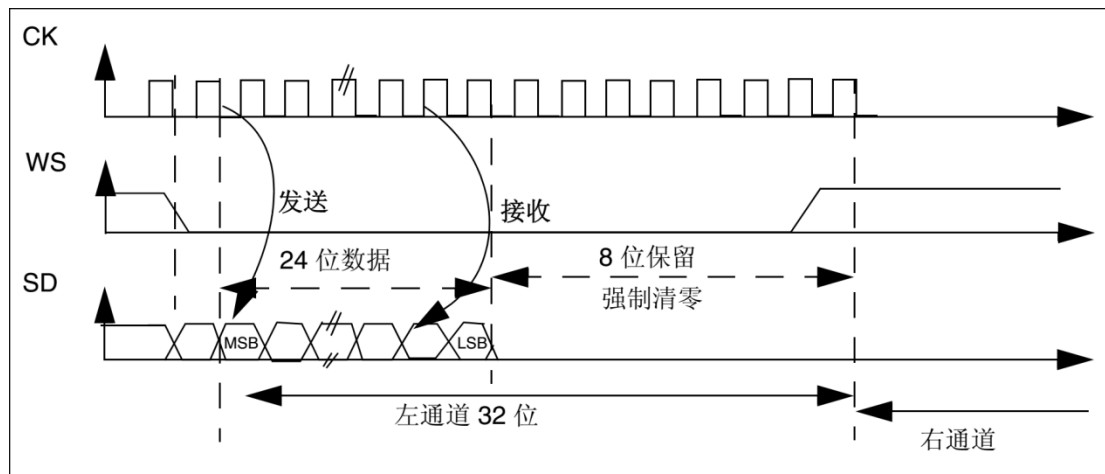


图 42-3 I2S Philips 标准 24bit 传输

2. 左对齐标准

在 WS 发生翻转同时开始传输数据，参考图 42-4，为 24bit 数据封装在 32bit 帧传输波形。该标准较少使用。注意此时 WS 为 1 时，传输的是左声道数据，这刚好与 I2S Philips 标准相反。

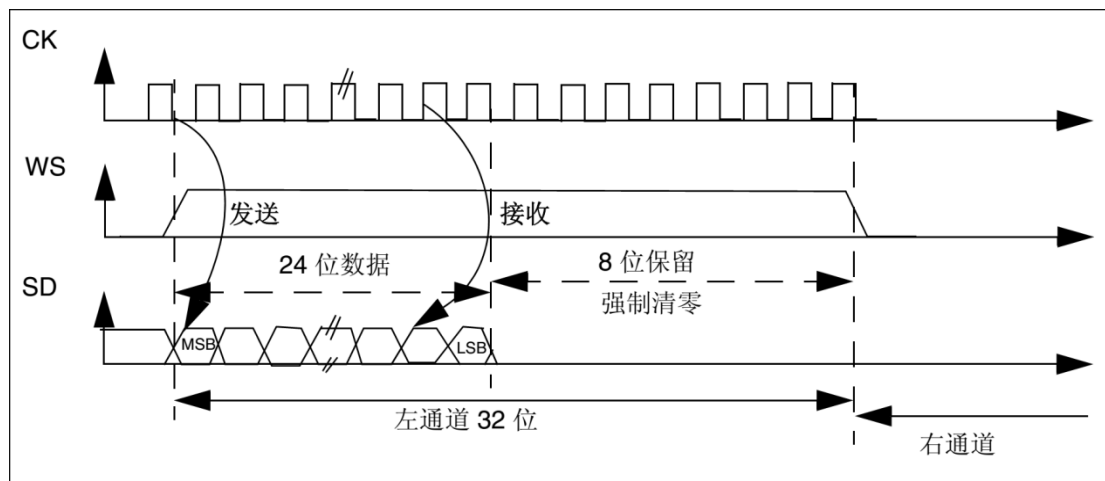


图 42-4 左对齐标准 24bit 传输

3. 右对齐标准

与左对齐标准类似，参考图 42-5，为 24bit 数据封装在 32bit 帧传输波形。

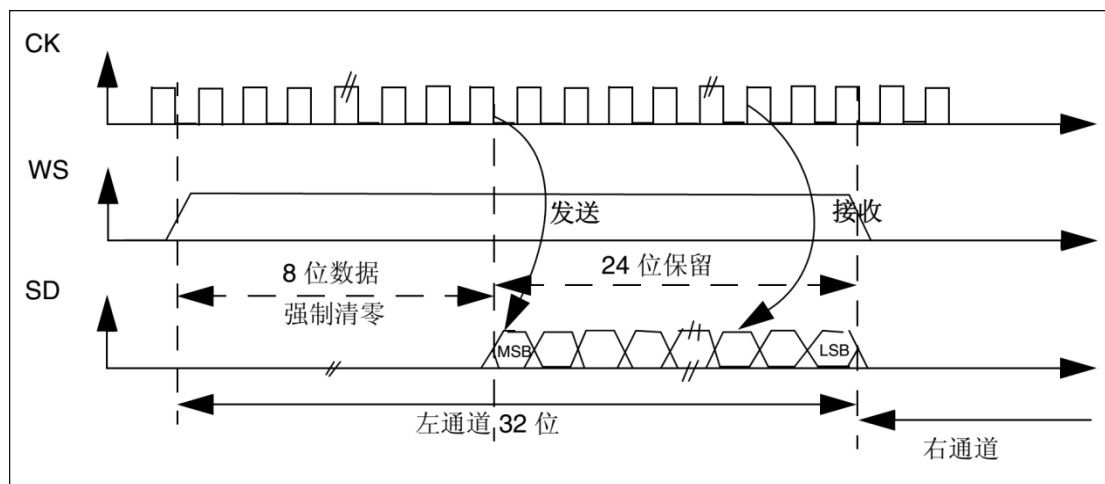


图 42-5 右对齐标准 24bit 传输

4. PCM 标准

PCM 即脉冲编码调制，模拟语音信号经过采样量化以及一定数据排列就是 PCM 了。WS 不再作为声道数据选择。它有两种模式，短帧模式和长帧模式，以 WS 信号高电平保持时间为判别依据，长帧模式保持 13 个 CK 周期，短帧模式只保持 1 个 CK 周期，可以通过相关寄存器位选择。如果有多通道数据是在一个 WS 周期内传输完成的，传完左声道数据就紧跟发送右声道数据。图 42-6 为单声道数据 16bit 扩展到 32bit 数据帧发送波形。

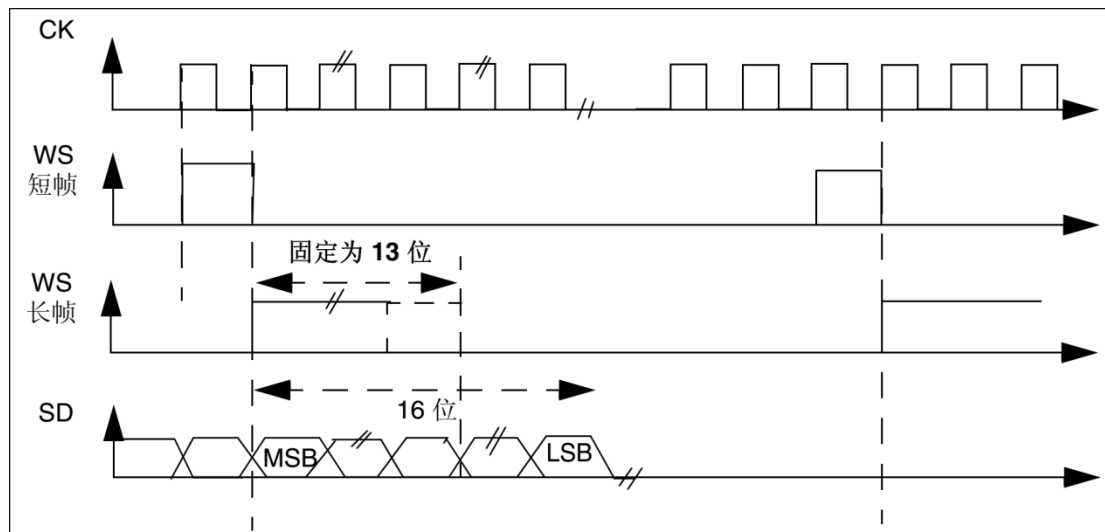


图 42-6 PCM 标准 16bit 传输

42.2 I2S 功能框图

STM32f4xx 系列控制器有两个 I2S，I2S2 和 I2S3，两个的资源是相互独立的，但分别与 SPI2 和 SPI3 共用大部分资源。这样 I2S2 和 SPI2 只能选择一个功能使用，I2S3 和 SPI3 相同道理。资源共用包括引脚共用和部分寄存器共用，当然也有部分是专用的。SPI 已经在之前相关章节做了详细讲解，建议先看懂 SPI 相关内容再学习 I2S。

控制器的 I2S 支持两种工作模式，主模式和从模式；主模式下使用自身时钟发生器生成通信时钟。I2S 功能框图参考图 42-7。

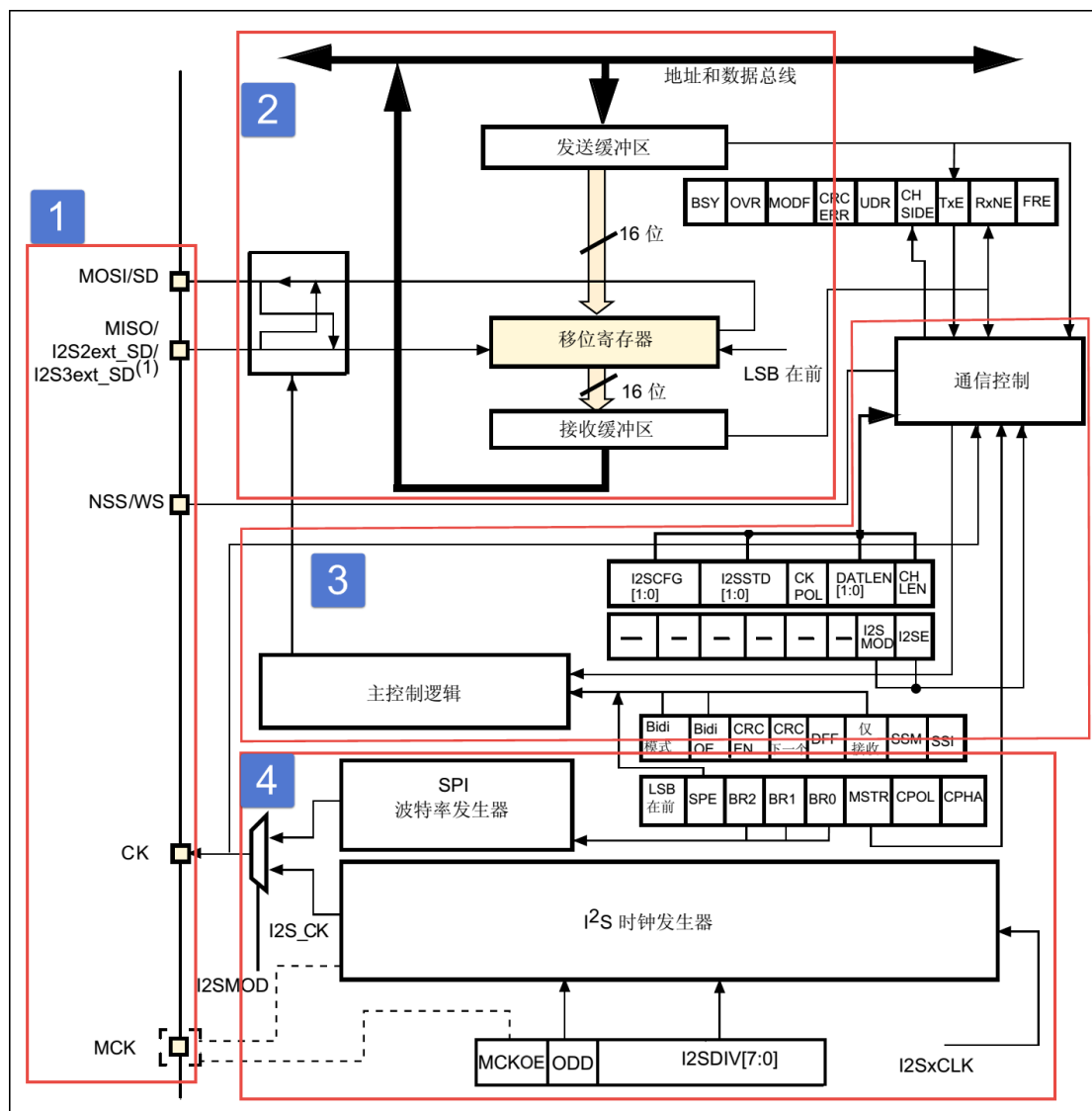


图 42-7 I2S 功能框图

1. 功能引脚

I2S 的 SD 映射到 SPI 的 MOSI 引脚，ext_SD 映射到 SPI 的 MISO 引脚，WS 映射到 SPI 的 NSS 引脚，CK 映射到 SPI 的 SCK 引脚。MCK 是 I2S 专用引脚，用于主模式下输出时钟或在从模式下输入时钟。I2S 时钟发生器可以由控制器内部时钟源分频产生，亦可采用 CKIN 引脚输入时钟分频得到，一般使用内部时钟源即可。控制器 I2S 引脚分布参考表 42-1。

表 42-1 STM32f4xx 系列控制器 I2S 引脚分布

引脚	I2S2	I2S3
SD	PC3/PB15/PI3	PC12/PD6/PB5
ext_SD	PC2/PB14/PI2	PC11/PB4
WS	PB12/PB9	PA4/PA15
CK	PB10/PB13/PD3	PC10/PB3
MCK	PC6	PC7
CKIN	PC9	

$$F_s = I2SxCLK / [(16 \times 2) \times ((2 \times I2SDIV) + ODD) \times 8]$$
 (通道帧宽度为 16bit 时)

$$F_s = I2SxCLK / [(32 \times 2) \times ((2 \times I2SDIV) + ODD) \times 4]$$
 (通道帧宽度为 32bit 时)

当禁止 MCK 时钟输出, 即 MCKOE=0 时, 采样频率计算如下:

$$F_s = I2SxCLK / [(16 \times 2) \times ((2 \times I2SDIV) + ODD)]$$
 (通道帧宽度为 16bit 时)

$$F_s = I2SxCLK / [(32 \times 2) \times ((2 \times I2SDIV) + ODD)]$$
 (通道帧宽度为 32bit 时)

42.3 WM8978 音频编解码器

WM8978 是一个低功耗、高质量的立体声多媒体数字信号编解码器。它主要应用于便携式应用。它结合了立体声差分麦克风的前置放大与扬声器、耳机和差分、立体声线输出的驱动, 减少了应用时必需的外部组件, 比如不需要单独的麦克风或者耳机的放大器。

高级的片上数字信号处理功能, 包含一个 5 路均衡功能, 一个用于 ADC 和麦克风或者线路输入之间的混合信号的电平自动控制功能, 一个纯粹的录音或者重放的数字限幅功能。另外在 ADC 的线路上提供了一个数字滤波的功能, 可以更好的应用滤波, 比如“减少风噪声”。

WM8978 可以被应用为一个主机或者一个从机。基于共同的参考时钟频率, 比如 12MHz 和 13MHz, 内部的 PLL 可以为编解码器提供所有需要的音频时钟。与 STM32 控制器连接使用, STM32 一般作为主机, WM8978 作为从机。

图 42-9 为 WM8978 芯片内部结构示意图, 参考来自《WM8978_v4.5》。该图给人的第一印象感觉就是很复杂, 密密麻麻很多内容, 特别有很多“开关”。实际上, 每个开关对应着 WM8978 内部寄存器的一个位, 通过控制寄存器的就可以控制开关的状态。

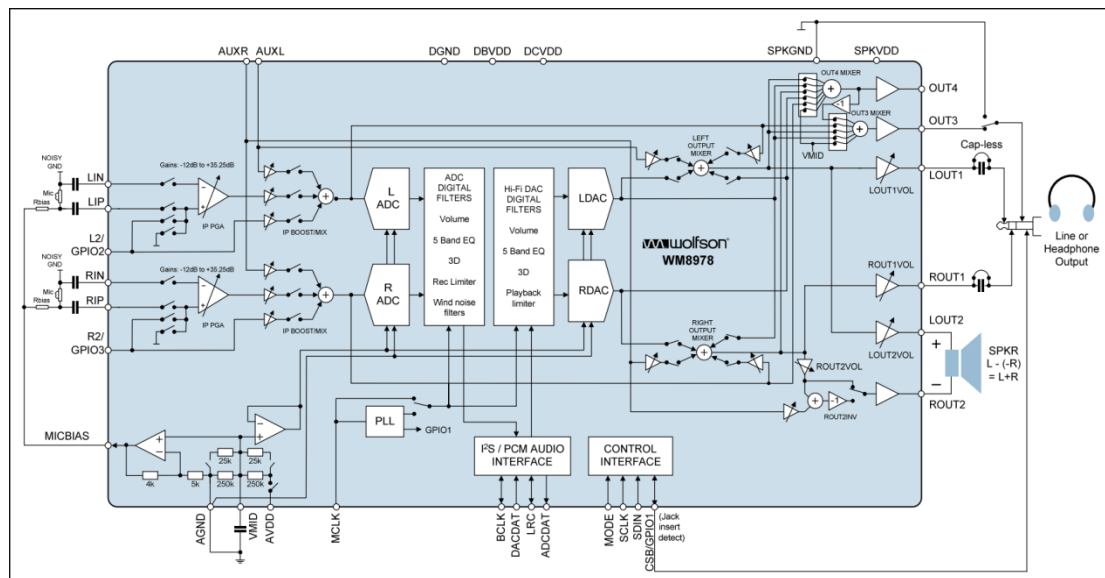


图 42-9 WM8978 内部结构

1. 输入部分

WM8978 结构图的左边部分是输入部分, 可用于模拟声音输入, 即用于录音输入。有三个输入接口, 一个是由 LIN 和 LIP、RIN 和 RIP 组合而成的伪差分立体声麦克风输入,

一个是由 L2 和 R2 组合的立体声麦克风输入，还有一个是由 AUXL 和 AUXR 组合的线输入或用来传输告警声的输入。

2. 输出部分

WM8978 结构图的右边部分是声音放大输出部分，LOUT1 和 ROUT1 用于耳机驱动，LOUT2 和 ROUT2 用于扬声器驱动，OUT3 和 OUT4 也可以配置成立体声线输出，OUT4 也可以用于提供一个左右声道的单声道混合。

3. ADC 和 DAC

WM8978 结构图的中边部分是芯片核心内容，处理声音的 AD 和 DA 转换。ADC 部分对声音输入进行处理，包括 ADC 滤波处理、音量控制、输入限幅器/电平自动控制等等。DAC 部分控制声音输出效果，包括 DAC5 路均衡器、DAC 3D 放大、DAC 输出限幅以及音量控制等等处理。

4. 通信接口

WM8978 有两个通信接口，一个是数字音频通信接口，另外一个控制接口。音频接口是采用 I2S 接口，支持左对齐、右对齐和 I2S 标准模式，以及 DSP 模式 A 和模拟 B。控制接口用于控制器发送控制命令配置 WM8978 运行状态，它提供 2 线或 3 线控制接口，对于 STM32 控制器，我们选择 2 线接口方式，它实际就是 I2C 总线方式，其芯片地址固定为 0011010。通过控制接口可以访问 WM8978 内部寄存器，实现芯片工作环境配置，总共有 58 个寄存器，表示为 R0 至 R57，限于篇幅问题这里不再深入探究，每个寄存器意义参考《WM8978_v4.5》了解。

WM8978 寄存器是 16bit 长度，高 7 位([15:9]bit)用于表示寄存器地址，低 9 为有实际意义，比如对于图 42-9 中的某个开关。所以在控制器向芯片发送控制命令时，必须传输长度为 16bit 的指令，芯片会根据接收命令高 7 位值寻址。

5. 其他部分

WM8978 作为主从机都必须对时钟进行管理，由内部 PLL 单元控制。另外还有电源管理单元。

42.4 WAV 格式文件

WAV 是微软公司开发的一种音频格式文件，用于保存 Windows 平台的音频信息资源，它符合资源互换文件格式(Resource Interchange File Format, RIFF)文件规范。标准格式化的 WAV 文件和 CD 格式一样，也是 44.1K 的取样频率，16 位量化数字，因此在声音文件质量和 CD 相差无几！WAVE 是录音时用的标准的 WINDOWS 文件格式，文件的扩展名为“WAV”，数据本身的格式为 PCM 或压缩型，属于无损音乐格式的一种。

42.4.1 RIFF 文件规范

RIFF 有不同数量的 chunk(区块)组成, 每个 chunk 由“标识符”、“数据大小”和“数据”三个部分组成, “标识符”和“数据大小”都是占用 4 个字节空间。简单 RIFF 格式文件结构参考图 42-10。最开始是 ID 为“RIFF”的 chunk, Size 为“RIFF” chunk 数据字节长度, 所以总文件大小为 Size+8。一般来说, chunk 不允许内部再包含 chunk, 但有两个例外, ID 为“RIFF”和“LIST”的 chunk 却是允许。对此“RIFF”在其“数据”首 4 个字节用来存放“格式标识码(Form Type)”, “LIST”则对应“LIST Type”。

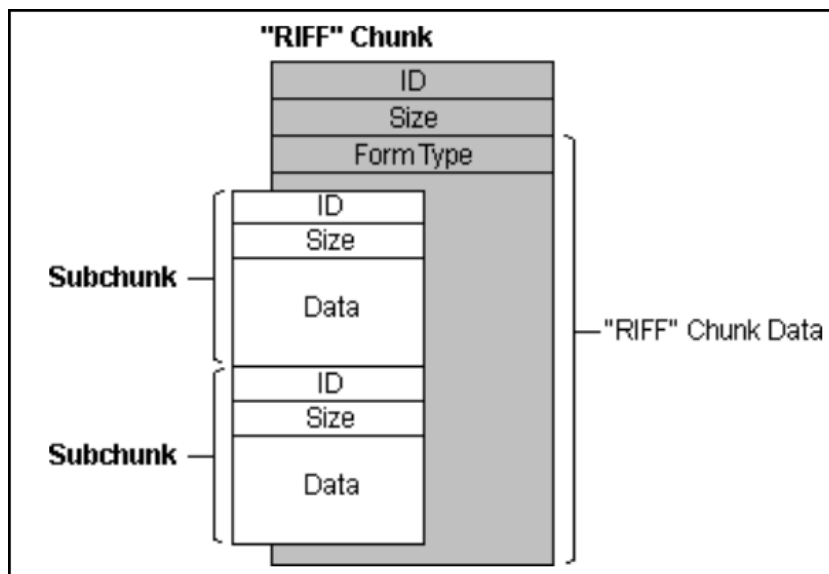


图 42-10 RIFF 文件格式结构

42.4.2 WAVE 文件

WAVE 文件是非常简单的一种 RIFF 文件, 其“格式标识码”定义为 WAVE。RIFF chunk 包括两个子 chunk, ID 分别为 fmt 和 data, 还有一个可选的 fact chunk。Fmt chunk 用于表示音频数据的属性, 包括编码方式、声道数目、采样频率、每个采样需要的 bit 数等等信息。fact chunk 是一个可选 chunk, 一般当 WAVE 文件由某些软件转化而成就包含 fact chunk。data chunk 包含 WAVE 文件的数字化波形声音数据。WAVE 整体结构如表 42-2。

表 42-2 WAVE 文件结构

标识码(“RIFF”)
数据大小
格式标识码(“WAVE”)
“fmt”
“fmt”块数据大小
“fmt”数据
“fact”(可选)
“fact”块数据大小
“fact”数据
“data”

声音数据大小
声音数据

data chunk 是 WAVE 文件主体部分，包含声音数据，一般有两个编码格式：PCM 和 ADPCM，ADPCM(自适应差分脉冲编码调制)属于有损压缩，现在几乎不用，绝大部分 WAVE 文件是 PCM 编码。PCM 编码声音数据可以说是在“数字音频技术”介绍的源数据，主要参数是采样频率和量化位数。

表 42-3 为量化位数为 16bit 时不同声道数据在 data chunk 数据排列格式。

表 42-3 16bit 声音数据格式

单声道	采样一		采样二		
	低字节	高字节	低字节	高字节	
双声道	采样一				
	左声道		右声道		
	低字节	高字节	低字节	高字节	

42.4.3 WAVE 文件实例分析

利用 winhex 工具软件可以非常方便以十六进制查看文件，图 42-11 为名为“张国荣-一盏小明灯.wav”文件使用 winhex 工具打开的部分界面截图。这部分截图是 WAVE 文件头部分，声音数据部分数据量非常大，有兴趣可以使用 winhex 查看。

Offset	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	
00000000	52	49	46	46	F4	FE	83	01	57	41	56	45	66	6D	74	20	RIFF 酷? WAVEfmt
00000010	10	00	00	00	01	00	02	00	44	AC	00	00	10	B1	02	00	D?...?..
00000020	04	00	10	00	64	61	74	61	48	FE	83	01	00	00	00	00	dataH??
00000030	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	

图 42-11 WAV 文件头实例

下面对文件头进行解读，参考表 42-4。

表 42-4 WAVE 文件格式说明

	偏移地址	字节数	数据类型	十六进制源码	内容
文件头	00H	4	char	52 49 46 46	“RIFF”标识符
	04H	4	long int	F4 FE 83 01	文件长度：0x0183FEF4(注意顺序)
	08H	4	char	57 41 56 45	“WAVE”标识符
	0CH	4	char	66 6D 74 20	“fmt”，最后一位为空格
	10H	4	long int	10 00 00 00	fmt chunk 大小：0x10
	14H	2	int	01 00	编码格式：0x01 为 PCM。
	16H	2	int	02 00	声道数目：0x01 为单声道，0x02 为双声道
	18H	4	int	44 AC 00 00	采样频率(每秒样本数)：0xAC44(44100)
	1CH	4	long int	10 B1 02 00	每秒字节数：0x02B110，等于声道数*采样频率*量化位数/8
	20H	2	int	04 00	每个采样点字节数：0x04，等于声道数*量化位数/8
	22H	2	int	10 00	量化位数：0x10
	24H	4	char	64 61 74 61	“data”数据标识符

42.5 I2S 初始化结构体详解

标准库函数对 I2S 外设建立了一个初始化结构体 `I2S_InitTypeDef`。初始化结构体成员用于设置 I2S 工作环境参数,并由 I2S 相应初始化配置函数 `I2S_Init` 调用,这些设定参数将会设置 I2S 相应的寄存器,达到配置 I2S 工作环境的目的。

初始化结构体和初始化库函数配合使用是标准库精髓所在,理解了初始化结构体每个成员意义基本上就可以对该外设运用自如了。初始化结构体定义在 `stm32f4xx_spi.h` 文件中,初始化库函数定义在 `stm32f4xx_spi.c` 文件中,编程时我们可以结合这两个文件内注释使用。

I2S 初始化结构体用于配置 I2S 基本工作环境,比如 I2S 工作模式、通信标准选择等等。它被 `I2S_Init` 函数调用。

代码清单 42-1 I2S_InitTypeDef 结构体

```
1 typedef struct {  
2     uint16_t I2S_Mode;           // I2S 模式选择  
3     uint16_t I2S_Standard;       // I2S 标准选择  
4     uint16_t I2S_DataFormat;     // 数据格式  
5     uint16_t I2S_MCLKOutput;     // 主时钟输出使能  
6     uint32_t I2S_AudioFreq;      // 采样频率  
7     uint16_t I2S_CPOL;           // 空闲电平选择  
8 } I2S_InitTypeDef;
```

- (1) `I2S_Mode`: I2S 模式选择,可选主机发送、主机接收、从机发送以及从机接收模式,它设定 `SPI_I2SCFGR` 寄存器 `I2SCFG` 位的值。一般设置 STM32 控制器为主机模式,当播放声音时选择发送模式;当录制声音时选择接收模式。
- (2) `I2S_Standard`: 通信标准格式选择,可选 I2S Philips 标准、左对齐标准、右对齐标准、PCM 短帧标准或 PCM 长帧标准,它设定 `SPI_I2SCFGR` 寄存器 `I2SSTD` 位和 `PCMSYNC` 位的值。一般设置为 I2S Philips 标准即可。
- (3) `I2S_DataFormat`: 数据格式选择,设定有效数据长度和帧长度,可选标准 16bit 格式、扩展 16bit(32bit 帧长度)格式、24bit 格式和 32bit 格式,它设定 `SPI_I2SCFGR` 寄存器 `DATLEN` 位和 `CHLEN` 位的值。对应 16bit 数据长度可选 16bit 或 32bit 帧长度,其他都是 32bit 帧长度。
- (4) `I2S_MCLKOutput`: 主时钟输出使能控制,可选使能输出或禁止输出,它设定 `SPI_I2SPR` 寄存器 `MCKOE` 位的值。为提高系统性能一般使能主时钟输出。
- (5) `I2S_AudioFreq`: 采样频率设置,标准库提供采样频率选择,分别为 8kHz、11kHz、16kHz、22kHz、32kHz、44kHz、48kHz、96kHz、192kHz 以及默认 2Hz,它设定 `SPI_I2SPR` 寄存器的值。
- (6) `I2S_CPOL`: 空闲状态的 CK 线电平,可选高电平或低电平,它设定 `SPI_I2SCFGR` 寄存器 `CKPOL` 位的值。一般设置为电平即可。

42.6 录音与回放实验

WAV 格式文件在现阶段一般以无损音乐格式存在,音质可以达到 CD 格式标准。结合上一章 SD 卡操作内容,本实验通过 FatFS 文件系统函数从 SD 卡读取 WAV 格式文件数据,

最终实现声音录制功能。同样的道理，WAV 格式文件中的音频数据可以直接传输给 WM8978 芯片实现音乐播放，整个过程与声音录制工程相反。

STM32 控制器与 WM8978 通信可分为两部分驱动函数，一部分是 I2C 控制接口，另一部分是 I2S 音频数据接口。

bsp_wm8978.c 和 bsp_wm8978.h 两个是专门创建用来存放 WM8978 芯片驱动代码。

1. I2C 控制接口

WM8978 要正常工作并且实现符合我们的要求，我们必须对芯片相关寄存器进行必须要配置，STM32 控制器通过 I2C 接口与 WM8978 芯片控制接口连接。I2C 接口内容也已经在以前做了详细介绍，这里主要讲解 WM8978 的功能函数。

bsp_wm8978.c 文件中的 I2C_GPIO_Config 函数、I2C_Mode_Config 函数以及 wm8978_Init 函数用于 I2C 通信接口 GPIO 和 I2C 相关配置，属于常规配置可以参考 GPIO 和 I2C 章节理解，这里不再分析，代码具体见本章配套程序工程文件。

输入输出选择枚举

代码清单 42-2 输入输出选择枚举

```
1 /* WM8978 音频输入通道控制选项，可以选择多路，比如 MIC_LEFT_ON | LINE_ON */
2 typedef enum {
3     IN_PATH_OFF    = 0x00, /* 无输入 */
4     MIC_LEFT_ON    = 0x01, /* LIN,LIP 脚，MIC 左声道（接板载咪头） */
5     MIC_RIGHT_ON   = 0x02, /* RIN,RIP 脚，MIC 右声道（接板载咪头） */
6     LINE_ON        = 0x04, /* L2,R2 立体声输入（接板载耳机插座） */
7     AUX_ON         = 0x08, /* AUXL,AUXR 立体声输入（开发板没用到） */
8     DAC_ON         = 0x10, /* I2S 数据 DAC（CPU 产生音频信号） */
9     ADC_ON         = 0x20 /* 输入的音频馈入 WM8978 内部 ADC（I2S 录音） */
10 } IN_PATH_E;
11
12 /* WM8978 音频输出通道控制选项，可以选择多路 */
13 typedef enum {
14     OUT_PATH_OFF    = 0x00, /* 无输出 */
15     EAR_LEFT_ON     = 0x01, /* LOUT1 耳机左声道（接板载耳机插座） */
16     EAR_RIGHT_ON    = 0x02, /* ROUT1 耳机右声道（接板载耳机插座） */
17     SPK_ON          = 0x04, /* LOUT2 和 ROUT2 反相输出单声道（开发板没用到） */
18     OUT3_4_ON       = 0x08, /* OUT3 和 OUT4 输出单声道音频（开发板没用到） */
19 } OUT_PATH_E;
```

IN_PATH_E 和 OUT_PATH_E 枚举了 WM8978 芯片可用的声音输入源和输出端口，具体到开发板，如果进行录音功能，设置输入源为(MIC_RIGHT_ON|ADC_ON)或(LINE_ON|ADC_ON)，设置输出端口为 OUT_PATH_OFF 或(EAR_LEFT_ON | EAR_RIGHT_ON)；对于音乐播放功能，设置输入源为 DAC_ON，设置输出端口为(EAR_LEFT_ON | EAR_RIGHT_ON)。

宏定义

代码清单 42-3 宏定义

```
1 /* 定义最大音量 */
2 #define VOLUME_MAX
3 #define VOLUME_STEP
4
5 /* 定义最大 MIC 增益 */
6 #define GAIN_MAX
63 /* 最大音量 */
1 /* 音量调节步长 */
63 /* 最大增益 */
```



```

7 #define GAIN_STEP 1 /* 增益步长 */
8
9 /* STM32 I2C 快速模式 */
10 #define WM8978_I2C_Speed 400000
11 /* WM8978 I2C 从机地址 */
12 #define WM8978_SLAVE_ADDRESS 0x34
13
14 /*I2C 接口*/
15 #define WM8978_I2C I2C1
16 #define WM8978_I2C_CLK RCC_APB1Periph_I2C1
17
18 #define WM8978_I2C_SCL_PIN GPIO_Pin_8
19 #define WM8978_I2C_SCL_GPIO_PORT GPIOB
20 #define WM8978_I2C_SCL_GPIO_CLK RCC_AHB1Periph_GPIOB
21 #define WM8978_I2C_SCL_SOURCE GPIO_PinSource8
22 #define WM8978_I2C_SCL_AF GPIO_AF_I2C1
23
24 #define WM8978_I2C_SDA_PIN GPIO_Pin_9
25 #define WM8978_I2C_SDA_GPIO_PORT GPIOB
26 #define WM8978_I2C_SDA_GPIO_CLK RCC_AHB1Periph_GPIOB
27 #define WM8978_I2C_SDA_SOURCE GPIO_PinSource9
28 #define WM8978_I2C_SDA_AF GPIO_AF_I2C1
29
30 /*等待超时时间*/
31 #define WM8978_I2C_FLAG_TIMEOUT ((uint32_t)0x4000)
32 #define WM8978_I2C_LONG_TIMEOUT ((uint32_t)(10 * WM8978_I2C_FLAG_TIMEOUT))

```

WM8978 声音调节有一定的范围限制，比如 R52(LOUT1 Volume Control)的 LOUT1VOL[5:0]位用于设置 LOUT1 的音量大小，可赋值范围为 0~63。WM8978 包含可调节的输入麦克风 PGA 增益，可对每个外部输入端口可单独设置增益大小，比如 R45(Left Channel input PGA volume control)的 INPPGAVOL[5:0]位用于设置左通道输入增益音量，最大可设置值为 63。

WM8978 控制接口被设置为 I2C 模式，其地址固定为 0011010，为方便使用，直接定义为 0x34。

最后定义 I2C 通信超时等待时间。

WM8978 寄存器写入

代码清单 42-4 WM8978 寄存器写入

```

1 static uint8_t WM8978_I2C_WriteRegister(uint8_t RegisterAddr,
2                                           uint16_t RegisterValue)
3 {
4     /* Start the config sequence */
5     I2C_GenerateSTART(WM8978_I2C, ENABLE);
6
7     /* Test on EV5 and clear it */
8     WM8978_I2CTimeout = WM8978_I2C_FLAG_TIMEOUT;
9     while (!I2C_CheckEvent(WM8978_I2C, I2C_EVENT_MASTER_MODE_SELECT)) {
10         if ((WM8978_I2CTimeout-- == 0))
11             return WM8978_I2C_TIMEOUT_UserCallback();
12     }
13
14     /* Transmit the slave address and enable writing operation */
15     I2C_Send7bitAddress(WM8978_I2C, WM8978_SLAVE_ADDRESS,
16                         I2C_Direction_Transmitter);
17
18     /* Test on EV6 and clear it */
19     WM8978_I2CTimeout = WM8978_I2C_FLAG_TIMEOUT;
20     while (!I2C_CheckEvent(WM8978_I2C,
21                             I2C_EVENT_MASTER_TRANSMITTER_MODE_SELECTED)) {
22         if ((WM8978_I2CTimeout-- == 0))
23             return WM8978_I2C_TIMEOUT_UserCallback();

```



```
24     }
25
26     /* Transmit the first address for write operation */
27     I2C_SendData(WM8978_I2C,
28                 ((RegisterAddr << 1) & 0xFE) | ((RegisterValue >> 8) & 0x1));
29
30     /* Test on EV8 and clear it */
31     WM8978_I2CTimeout = WM8978_I2C_FLAG_TIMEOUT;
32     while (!I2C_CheckEvent(WM8978_I2C,
33                            I2C_EVENT_MASTER_BYTE_TRANSMITTING)) {
34         if ((WM8978_I2CTimeout-- == 0)
35             return WM8978_I2C_TIMEOUT_UserCallback();
36     }
37
38     /* Prepare the register value to be sent */
39     I2C_SendData(WM8978_I2C, RegisterValue&0xff);
40
41     /*!< Wait till all data have been physically transferred on the bus */
42     WM8978_I2CTimeout = WM8978_I2C_LONG_TIMEOUT;
43     while (!I2C_GetFlagStatus(WM8978_I2C, I2C_FLAG_BTF)) {
44         if ((WM8978_I2CTimeout-- == 0)
45             WM8978_I2C_TIMEOUT_UserCallback();
46     }
47
48     /* End the configuration sequence */
49     I2C_GenerateSTOP(WM8978_I2C, ENABLE);
50
51     /* Return the verifying value: 0 (Passed) or 1 (Failed) */
52     return 1;
53 }
```

WM8978_I2C_WriteRegister 用于向 WM8978 芯片寄存器写入数值，达到配置芯片工作环境，函数有两个形参，一个是寄存器地址，可设置范围为 0~57；另外一个为寄存器值，WM8978 芯片寄存器总共有 16bit，前 7bit 用于寻址，后 9 位才是数据，这里寄存器值形参使用 uint16_t 类型，只有低 9 位有效。

使用 I2C 通信，首先使用 I2C_Send7bitAddress 函数选定 WM8978 芯片，接下来需要两次调用 I2C_SendData 函数发送两次数据，因为 I2C 数据发送一次只能发送 8bit 数据，为此需要把 RegisterValue 变量的第 9bit 整合到 RegisterAddr 变量的第 0 位先发送，接下来再发送 RegisterValue 变量的低 8bit 数据。

函数中还添加了 I2C 通信超时等待功能，防止出错时卡死。

WM8978 寄存器读取

WM8978 芯片是从硬件上选择 I2C 通信模式，该模式是只写的，STM32 控制器无法读取 WM8978 寄存器内容，但程序有时需要用到寄存器内容，为此我们创建了一个存放 WM8978 所有寄存器值的数组，在系统复位是将数组内容设置为 WM8978 缺省值，然后在每次修改寄存器内容时同步更新该数组内容，这样可以达到该数组与 WM8978 寄存器内容相等的效果，参考代码清单 42-5。

代码清单 42-5 WM8978 寄存器值缓冲区和读取

```
1 /*
2  wm8978 寄存器缓存
3  由于 WM8978 的 I2C 两线接口不支持读取操作，因此寄存器值缓存在内存中，
4  当写寄存器时同步更新缓存，读寄存器时直接返回缓存中的值。
5  寄存器 MAP 在 WM8978 (V4.5_2011).pdf 的第 89 页，寄存器地址是 7bit，寄存器数据是 9bit
6  */
7 static uint16_t wm8978_RegCash[] = {
8     0x000, 0x000, 0x000, 0x000, 0x050, 0x000, 0x140, 0x000,
```

```

9      0x000, 0x000, 0x000, 0x0FF, 0x0FF, 0x000, 0x100, 0x0FF,
10     0x0FF, 0x000, 0x12C, 0x02C, 0x02C, 0x02C, 0x02C, 0x000,
11     0x032, 0x000, 0x000, 0x000, 0x000, 0x000, 0x000, 0x000,
12     0x038, 0x00B, 0x032, 0x000, 0x008, 0x00C, 0x093, 0x0E9,
13     0x000, 0x000, 0x000, 0x000, 0x003, 0x010, 0x010, 0x100,
14     0x100, 0x002, 0x001, 0x001, 0x039, 0x039, 0x039, 0x039,
15     0x001, 0x001
16 };
17
18 /**
19  * @brief 从 cash 中读回读回 wm8978 寄存器
20  * @param _ucRegAddr : 寄存器地址
21  * @retval 寄存器值
22  */
23 static uint16_t wm8978_ReadReg(uint8_t _ucRegAddr)
24 {
25     return wm8978_RegCash[_ucRegAddr];
26 }
27
28 /**
29  * @brief 写 wm8978 寄存器
30  * @param _ucRegAddr: 寄存器地址
31  * @param _usValue: 寄存器值
32  * @retval 0: 写入失败
33  *         1: 写入成功
34  */
35 static uint8_t wm8978_WriteReg(uint8_t _ucRegAddr, uint16_t _usValue)
36 {
37     uint8_t res;
38     res=WM8978_I2C_WriteRegister(_ucRegAddr, _usValue);
39     wm8978_RegCash[_ucRegAddr] = _usValue;
40     return res;
41 }

```

wm8978_WriteReg 实现向 WM8978 寄存器写入数据并修改缓冲区内容。

输出音量修改与读取

代码清单 42-6 音量修改与读取

```

1  /**
2   * @brief 修改输出通道 1 音量
3   * @param _ucVolume : 音量值, 0-63
4   * @retval 无
5   */
6 void wm8978_SetOUT1Volume(uint8_t _ucVolume)
7 {
8     uint16_t regL;
9     uint16_t regR;
10
11     if (_ucVolume > VOLUME_MAX) {
12         _ucVolume = VOLUME_MAX;
13     }
14     regL = _ucVolume;
15     regR = _ucVolume;
16     /*
17      R52 LOUT1 Volume control
18      R53 ROUT1 Volume control
19     */
20     /* 先更新左声道缓存值 */
21     wm8978_WriteReg(52, regL | 0x00);
22
23     /* 再同步更新左右声道的音量 */
24     /* 0x180 表示 在音量为 0 时再更新, 避免调节音量出现的“嘎哒”声 */
25     wm8978_WriteReg(53, regR | 0x100);
26 }
27

```

```
28 /**
29  * @brief  读取输出通道 1 音量
30  * @param  无
31  * @retval 当前音量值
32  */
33 uint8_t wm8978_ReadOUT1Volume(void)
34 {
35     return (uint8_t) (wm8978_ReadReg(52) & 0x3F );
36 }
37
38 /**
39  * @brief  输出静音.
40  * @param  _ucMute: 模式选择
41  *          @arg 1: 静音
42  *          @arg 0: 取消静音
43  * @retval 无
44  */
45 void wm8978_OutMute(uint8_t _ucMute)
46 {
47     uint16_t usRegValue;
48     if (_ucMute == 1) { /* 静音 */
49         usRegValue = wm8978_ReadReg(52); /* Left Mixer Control */
50         usRegValue |= (1u << 6);
51         wm8978_WriteReg(52, usRegValue);
52
53         usRegValue = wm8978_ReadReg(53); /* Left Mixer Control */
54         usRegValue |= (1u << 6);
55         wm8978_WriteReg(53, usRegValue);
56
57         usRegValue = wm8978_ReadReg(54); /* Right Mixer Control */
58         usRegValue |= (1u << 6);
59         wm8978_WriteReg(54, usRegValue);
60
61         usRegValue = wm8978_ReadReg(55); /* Right Mixer Control */
62         usRegValue |= (1u << 6);
63         wm8978_WriteReg(55, usRegValue);
64     } else { /* 取消静音 */
65         usRegValue = wm8978_ReadReg(52);
66         usRegValue &= ~(1u << 6);
67         wm8978_WriteReg(52, usRegValue);
68
69         usRegValue = wm8978_ReadReg(53); /* Left Mixer Control */
70         usRegValue &= ~(1u << 6);
71         wm8978_WriteReg(53, usRegValue);
72
73         usRegValue = wm8978_ReadReg(54);
74         usRegValue &= ~(1u << 6);
75         wm8978_WriteReg(54, usRegValue);
76
77         usRegValue = wm8978_ReadReg(55); /* Left Mixer Control */
78         usRegValue &= ~(1u << 6);
79         wm8978_WriteReg(55, usRegValue);
80     }
81 }
```

`wm8978_SetOUT1Volume` 函数用于修改 OUT1 通道的音量大小，有一个形参用于指示音量大小，要求范围为 0~63。这里同时更新 OUT1 的左右两个声道音量，WM8978 芯片的 R52 和 R53 分别用于设置 OUT1 的左声道和右声道音量，具体位段意义参考表 42-5。
`wm8978_SetOUT1Volume` 函数会同时修改 WM8978 寄存器缓存区 `wm8978_RegCash` 数组内容。

表 42-5 OUT1 音量控制寄存器

寄存器地址	位	默认值	描述
-------	---	-----	----

R52(LOUT1 Volume Control)	8	不锁存	直到一个 1 写入到 HPVU 才更新 LOUT1 和 ROUT1 音量
	7	0	耳机音量零交叉使能：0=仅仅在零交叉时改变增益；1=立即改变增益
	6	0	左耳机输出消声：0=正常操作；1=消声
	5:0	11101	左耳机输出驱动： 000000=-57dB ... 111001=0dB ... 111111=+6dB
R53(ROUT1 Volume Control)	8	不锁存	直到一个 1 写入到 HPVU 才更新 LOUT1 和 ROUT1 音量
	7	0	耳机音量零交叉使能：0=仅仅在零交叉时改变增益；1=立即改变增益
	6	0	左耳机输出消声：0=正常操作；1=消声
	5:0	11101	右耳机输出驱动： 000000=-57dB ... 111001=0dB ... 111111=+6dB

另外，wm8978_SetOUT2Volume 用于设置 OUT2 的音量，程序结构与 wm8978_SetOUT1Volume 相同，只是对应修改 R54 和 R55。

wm8978_ReadOUT1Volume 函数用于读取 OUT1 的音量，它实际就是读取 wm8978_RegCash 数组对应元素内容。

wm8978_OutMute 用于静音控制，它有一个形参用于设置静音效果，如果为 1 则为开启静音，如果为 0 则取消静音。静音控制是通过 R52 和 R53 的第 6 位实现的，在进入静音模式时需要先保存 OUT1 和 OUT2 的音量大小，然后在退出静音模式时就可以正确返回到静音前 OUT1 和 OUT2 的配置。

输入增益调整

代码清单 42-7 输入增益调整

```

1 /**
2  * @brief 设置增益
3  * @param _ucGain : 增益值, 0-63
4  * @retval 无
5  */
6 void wm8978_SetMicGain(uint8_t _ucGain)
7 {
8     if (_ucGain > GAIN_MAX) {
9         _ucGain = GAIN_MAX;
10    }
11
12    /* PGA 音量控制 R45, R46
13       Bit8 INPPGAUPDATE
14       Bit7 INPPGAZCL  过零再更改
15       Bit6 INPPGAMUTEL PGA 静音
16       Bit5:0 增益值, 010000 是 0dB
17    */
18    wm8978_WriteReg(45, _ucGain);
19    wm8978_WriteReg(46, _ucGain | (1 << 8));
20 }
```

```
21
22
23 /**
24  * @brief 设置 Line 输入通道的增益
25  * @param _ucGain : 音量值, 0~7. 7 最大, 0 最小。 可衰减可放大。
26  * @retval 无
27  */
28 void wm8978_SetLineGain(uint8_t _ucGain)
29 {
30     uint16_t usRegValue;
31
32     if (_ucGain > 7) {
33         _ucGain = 7;
34     }
35
36     /*
37      Mic 输入信道的增益由 PGABOOSTL 和 PGABOOSTR 控制
38      Aux 输入信道的输入增益由 AUXL2BOOSTVO[2:0] 和 AUXR2BOOSTVO[2:0] 控制
39      Line 输入信道的增益由 LIP2BOOSTVOL[2:0] 和 RIP2BOOSTVOL[2:0] 控制
40     */
41     /* R47 (左声道), R48 (右声道), MIC 增益控制寄存器
42      R47 (R48 定义与此相同)
43      B8 PGABOOSTL=1,0 表示 MIC 信号直通无增益, 1 表示 MIC 信号+20dB 增益 (通过自举电路)
44      B7 = 0, 保留
45      B6:4 L2_2BOOSTVOL=x, 0 表示禁止, 1-7 表示增益-12dB~ +6dB (可以衰减也可以放大)
46      B3 = 0, 保留
47      B2:0 AUXL2BOOSTVOL=x, 0 表示禁止, 1-7 表示增益-12dB~+6dB (可以衰减也可以放大)
48     */
49
50     usRegValue = wm8978_ReadReg(47);
51     usRegValue &= 0x8F; /* 将 Bit6:4 清 0 1000 1111 */
52     usRegValue |= (_ucGain << 4);
53     wm8978_WriteReg(47, usRegValue); /* 写左声道输入增益控制寄存器 */
54
55     usRegValue = wm8978_ReadReg(48);
56     usRegValue &= 0x8F; /* 将 Bit6:4 清 0 1000 1111 */
57     usRegValue |= (_ucGain << 4);
58     wm8978_WriteReg(48, usRegValue); /* 写右声道输入增益控制寄存器 */
59 }
```

wm8978_SetMicGain 用于设置麦克风输入的增益, 可以设置增强或减弱输入效果, 比如对于部分声音源本身就是比较微弱, 我们就可以设置放大该信号, 从而得到合适的录制效果, 该函数主要通过设置 R45 和 R46 实现, 可设置的范围为 0~63, 默认值为 16, 没有增益效果。

wm8978_SetLineGain 用于设置 LINE 输入的增益, 对应芯片的 L2 和 R2 引脚组合的输入, 开发板使用耳机插座引出拓展。它通过设置 R47 和 R48 寄存器实现, 可设置范围为 0~7, 默认值为 0, 没有增益效果。

音频接口标准选择

代码清单 42-8 wm8978_CfgAudioIF 函数

```
1 /**
2  * @brief 配置 WM8978 的音频接口 (I2S)
3  * @param _usStandard : 接口标准,
4  * I2S_Standard_Phillips, I2S_Standard_MSB 或 I2S_Standard_LSB
5  * @param _ucWordLen : 字长, 16、24、32 (丢弃不常用的 20bit 格式)
6  * @retval 无
7  */
8 void wm8978_CfgAudioIF(uint16_t _usStandard, uint8_t _ucWordLen)
9 {
```

```
10     uint16_t usReg;
11
12     /* WM8978 (V4.5_2011).pdf 73 页, 寄存器列表 */
13
14     /* REG R4, 音频接口控制寄存器
15        B8      BCP  = x, BCLK 极性, 0 表示正常, 1 表示反相
16        B7      LRCP = x, LRC 时钟极性, 0 表示正常, 1 表示反相
17        B6:5    WL  = x, 字长, 00=16bit, 01=20bit, 10=24bit, 11=32bit
18                (右对齐模式只能操作在最大 24bit)
19        B4:3    FMT  = x, 音频数据格式, 00=右对齐, 01=左对齐, 10=I2S 格式, 11=PCM
20        B2      DACLRSWAP = x, 控制 DAC 数据出现在 LRC 时钟的左边还是右边
21        B1      ADCLRSWAP = x, 控制 ADC 数据出现在 LRC 时钟的左边还是右边
22        B0      MONO   = 0, 0 表示立体声, 1 表示单声道, 仅左声道有效
23    */
24    usReg = 0;
25    if (_usStandard == I2S_Standard_Phillips) { /* I2S 飞利浦标准 */
26        usReg |= (2 << 3);
27    } else if (_usStandard == I2S_Standard_MSB) { /* MSB 对齐标准(左对齐) */
28        usReg |= (1 << 3);
29    } else if (_usStandard == I2S_Standard_LSB) { /* LSB 对齐标准(右对齐) */
30        usReg |= (0 << 3);
31    } else { /* PCM 标准(16 位通道帧上带长或短帧同步或者 16 位数据帧扩展为 32 位通道帧) */
32        usReg |= (3 << 3);
33    }
34
35    if (_ucWordLen == 24) {
36        usReg |= (2 << 5);
37    } else if (_ucWordLen == 32) {
38        usReg |= (3 << 5);
39    } else {
40        usReg |= (0 << 5); /* 16bit */
41    }
42    wm8978_WriteReg(4, usReg);
43
44    /*
45        R6, 时钟产生控制寄存器
46        MS = 0, WM8978 被动时钟, 由 MCU 提供 MCLK 时钟
47    */
48    wm8978_WriteReg(6, 0x000);
49 }
```

wm8978_CfgAudioIF 函数用于设置 WM8978 芯片的音频接口标准, 它有两个形参, 第一个是标准选择, 可选 I2S Philips 标准(I2S_Standard_Phillips)、左对齐标准(I2S_Standard_MSB)以及右对齐标准(I2S_Standard_LSB); 另外一个形参是字长设置, 可选 16bit、24bit 以及 32bit, 较常用 16bit。它函数通过控制 WM8978 芯片 R4 实现, 最后还通过通用时钟控制寄存器 R6 设置芯片的 I2S 工作在从模式, 时钟线为输入时钟。

输入输出通道设置

代码清单 42-9 wm8978_CfgAudioPath 函数

```
1 void wm8978_CfgAudioPath(uint16_t _InPath, uint16_t _OutPath)
2 {
3     uint16_t usReg;
4     if ((_InPath == IN_PATH_OFF) && (_OutPath == OUT_PATH_OFF)) {
5         wm8978_PowerDown();
6         return;
7     }
8
9     /*
10        R1 寄存器 Power manage 1
11        Bit8    BUFDOPEN, Output stage 1.5xAVDD/2 driver enable
12        Bit7    OUT4MIXEN, OUT4 mixer enable

```



```
13      Bit6      OUT3MIXEN, OUT3 mixer enable
14      Bit5      PLEN .不用
15      Bit4      MICBEN ,Microphone Bias Enable (MIC 偏置电路使能)
16      Bit3      BIASEN ,Analogue amplifier bias control 必须设置为 1 模拟放大器才工作
17      Bit2      BUFIOEN , Unused input/output tie off buffer enable
18      Bit1:0     VMIDSEL, 必须设置为非 00 值模拟放大器才工作
19      */
20      usReg = (1 << 3) | (3 << 0);
21      if (_OutPath & OUT3_4_ON) { /* OUT3 和 OUT4 使能输出 */
22          usReg |= ((1 << 7) | (1 << 6));
23      }
24      if ((_InPath & MIC_LEFT_ON) || (_InPath & MIC_RIGHT_ON)) {
25          usReg |= (1 << 4);
26      }
27      wm8978_WriteReg(1, usReg); /* 写寄存器 */
28
29      /*****
30      /*          此处省略部分代码，具体参考工程文件          */
31      *****/
32
33      /* R10 寄存器 DAC Control
34      B8  0
35      B7  0
36      B6  SOFTMUTE, Softmute enable:
37      B5  0
38      B4  0
39      B3  DACOSR128, DAC oversampling rate: 0=64x (lowest power)
40                                          1=128x (best performance)
41      B2  AMUTE, Automute enable
42      B1  DACPOLR, Right DAC output polarity
43      B0  DACPOLL, Left DAC output polarity:
44      */
45      if (_InPath & DAC_ON) {
46          wm8978_WriteReg(10, 0);
47      }
48 }
```

wm8978_CfgAudioPath 函数用于配置声音输入输出通道，有两个形参，第一个形参用于设置输入源，可以使用 IN_PATH_E 枚举类型成员的一个或多个或运算结果；第二个形参用于设置输出通道，可以使用 OUT_PATH_E 枚举类型成员的一个或多个或运算结果。具体到开发板，如果进行录音功能，设置输入源为(MIC_RIGHT_ON|ADC_ON)或(LINE_ON|ADC_ON)，设置输出端口为 OUT_PATH_OFF 或(EAR_LEFT_ON | EAR_RIGHT_ON)；对于音乐播放功能，设置输入源为 DAC_ON，设置输出端口为(EAR_LEFT_ON | EAR_RIGHT_ON)。

wm8978_CfgAudioPath 函数首先判断输入参数合法性，如果输入出错直接调用函数 wm8978_PowerDown 进入低功耗模式，并退出。

接下来使用 wm8978_WriteReg 配置相关寄存器值。大致可分三个部分，第一部分是电源管理部分，主要涉及到 R1、R2 和 R3 三个寄存器，使用输入输出通道之前必须开启相关电源。第二部分是输入通道选择及相关配置，配置 R44 控制选择输入通道，R14 设置输入的高通滤波器功能，R27、R28、R29 和 R30 设置输入的可调陷波滤波器功能，R32、R33 和 R34 控制输入限幅器/电平自动控制(ALC)，R35 设置 ALC 噪声门限，R47 和 R48 设置通道增益参数，R15 和 R16 设置 ADC 数字音量，R43 设置 AUXR 功能。第三部分是输出通道选择及相关配置，控制 R49 选择输出通道，R50 和 R51 设置左右通道混合输出效果，

R56 设置 OUT3 混合输出效果, R57 设置 OUT4 混合输出效果, R11 和 R12 设置左右 DAC 数字音量, R10 设置 DAC 参数。

软件复位

代码清单 42-10 wm8978_Reset 函数

```
1 uint8_t wm8978_Reset(void)
2 {
3     /* wm8978 寄存器缺省值 */
4     const uint16_t reg_default[] = {
5         0x000, 0x000, 0x000, 0x000, 0x050, 0x000, 0x140, 0x000,
6         0x000, 0x000, 0x000, 0x0FF, 0x0FF, 0x000, 0x100, 0x0FF,
7         0x0FF, 0x000, 0x12C, 0x02C, 0x02C, 0x02C, 0x02C, 0x000,
8         0x032, 0x000, 0x000, 0x000, 0x000, 0x000, 0x000, 0x000,
9         0x038, 0x00B, 0x032, 0x000, 0x008, 0x00C, 0x093, 0x0E9,
10        0x000, 0x000, 0x000, 0x000, 0x003, 0x010, 0x010, 0x100,
11        0x100, 0x002, 0x001, 0x001, 0x039, 0x039, 0x039, 0x039,
12        0x001, 0x001
13    };
14    uint8_t res;
15    uint8_t i;
16
17    res=wm8978_WriteReg(0x00, 0);
18
19    for (i = 0; i < sizeof(reg_default) / 2; i++) {
20        wm8978_RegCash[i] = reg_default[i];
21    }
22    return res;
23 }
```

wm8978_Reset 函数用于软件复位 WM8978 芯片, 通过写入 R0 完成, 使其寄存器复位到缺省状态, 同时会更新寄存器缓冲区数组 wm8978_RegCash 恢复到缺省状态。

2. I2S 控制接口

WM8978 集成 I2S 音频接口, 用于与外部设备进行数字音频数据传输, 芯片 I2S 接口属性通过 wm8978_CfgAudioIF 函数配置。STM32 控制器与 WM8978 进行音频数据传输, 一般设置 STM32 控制器为主机模式, WM8978 作为从设备。

I2S_GPIO_Config 函数用于初始化 I2S 相关 GPIO, 具体参考工程文件。

I2S 工作模式配置

代码清单 42-11 I2Sx_Mode_Config 函数

```
1 void I2Sx_Mode_Config(const uint16_t _usStandard,
2                        const uint16_t _usWordLen, const uint32_t _usAudioFreq )
3 {
4     I2S_InitTypeDef I2S_InitStructure;
5     uint32_t n = 0;
6     FlagStatus status = RESET;
7     /**
8      * For I2S mode, make sure that either:
9      * - I2S PLL is configured using the functions RCC_I2SCLKConfig
10     *   (RCC_I2S2CLKSource_PLLI2S),
11     *   RCC_PLLI2SCmd(ENABLE) and RCC_GetFlagStatus(RCC_FLAG_PLLI2SRDY).
12     */
13     RCC_I2SCLKConfig(RCC_I2S2CLKSource_PLLI2S);
14     RCC_PLLI2SCmd(ENABLE);
15     for (n = 0; n < 500; n++) {
16         status = RCC_GetFlagStatus(RCC_FLAG_PLLI2SRDY);
17         if (status == 1)break;
18     }
```

```
19  /* 打开 I2S2 APB1 时钟 */
20  RCC_APB1PeriphClockCmd(WM8978_CLK, ENABLE);
21
22  /* 复位 SPI2 外设到缺省状态 */
23  SPI_I2S_DeInit(WM8978_I2Sx_SPI);
24
25  /* I2S2 外设配置 */
26  /* 配置 I2S 工作模式 */
27  I2S_InitStructure.I2S_Mode = I2S_Mode_MasterTx;
28  /* 接口标准 */
29  I2S_InitStructure.I2S_Standard = _usStandard;
30  /* 数据格式, 16bit */
31  I2S_InitStructure.I2S_DataFormat = _usWordLen;
32  /* 主时钟模式 */
33  I2S_InitStructure.I2S_MCLKOutput = I2S_MCLKOutput_Enable;
34  /* 音频采样频率 */
35  I2S_InitStructure.I2S_AudioFreq = _usAudioFreq;
36  I2S_InitStructure.I2S_CPOL = I2S_CPOL_Low;
37  I2S_Init(WM8978_I2Sx_SPI, &I2S_InitStructure);
38
39  /* 使能 SPI2/I2S2 外设 */
40  I2S_Cmd(WM8978_I2Sx_SPI, ENABLE);
41 }
```

I2Sx_Mode_Config 函数用于配置 STM32 控制器的 I2S 接口工作模式，它有三个形参，第一个为指定 I2S 接口标准，一般设置为 I2S Philips 标准，第二个为字长设置，一般设置为 16bit，第三个为采样频率，一般设置为 44KHz 既可得到高音质效果。

首先是时钟配置，使用 RCC_I2SCLKConfig 函数选择 I2S 时钟源，一般选择内部 PLLI2S 时钟，RCC_PLLI2SCmd 函数使能 PLLI2S 时钟，并等待时钟正常后开启 I2S 外设时钟。

接下来通过给 I2S_InitTypeDef 结构体类型变量赋值设置 I2S 工作模式，并由 I2S_Init 函数完成 I2S 基本工作环境配置。

最后，I2S_Cmd 函数用于使能 I2S。

I2S 数据发送(DMA 传输)

代码清单 42-12 I2Sx_TX_DMA_Init 函数

```
1 void I2Sx_TX_DMA_Init(const uint16_t *buffer0, const uint16_t *buffer1,
2                       const uint32_t num)
3 {
4     NVIC_InitTypeDef  NVIC_InitStructure;
5     DMA_InitTypeDef   DMA_InitStructure;
6
7     RCC_AHB1PeriphClockCmd(I2Sx_DMA_CLK, ENABLE); //DMA1 时钟使能
8
9     DMA_DeInit(I2Sx_TX_DMA_STREAM);
10    //等待 DMA1_Stream4 可配置
11    while (DMA_GetCmdStatus(I2Sx_TX_DMA_STREAM) != DISABLE) {}
12    //清空 DMA1_Stream4 上所有中断标志
13    DMA_ClearITPendingBit(I2Sx_TX_DMA_STREAM,
14        DMA_IT_FEIF4|DMA_IT_DMEIF4|DMA_IT_TEIF4|DMA_IT_HTIF4|DMA_IT_TCIF4);
15
16    /* 配置 DMA Stream */
17    //通道 0 SPIx_TX 通道
18    DMA_InitStructure.DMA_Channel = I2Sx_TX_DMA_CHANNEL;
19    //外设地址为: (u32) &SPI2->DR
20    DMA_InitStructure.DMA_PeripheralBaseAddr = (uint32_t) &WM8978_I2Sx_SPI->DR;
21
22    //DMA 存储器 0 地址
```

```
23 DMA_InitStructure.DMA_Memory0BaseAddr = (uint32_t)buffer0;
24 //存储器到外设模式
25 DMA_InitStructure.DMA_DIR = DMA_DIR_MemoryToPeripheral;
26 //数据传输量
27 DMA_InitStructure.DMA_BufferSize = num;
28 //外设非增量模式
29 DMA_InitStructure.DMA_PeripheralInc = DMA_PeripheralInc_Disable;
30 //存储器增量模式
31 DMA_InitStructure.DMA_MemoryInc = DMA_MemoryInc_Enable;
32 //外设数据长度:16 位
33 DMA_InitStructure.DMA_PeripheralDataSize = DMA_PeripheralDataSize_HalfWord;
34
35 //存储器数据长度: 16 位
36 DMA_InitStructure.DMA_MemoryDataSize = DMA_MemoryDataSize_HalfWord;
37 // 使用循环模式
38 DMA_InitStructure.DMA_Mode = DMA_Mode_Circular;
39 //高优先级
40 DMA_InitStructure.DMA_Priority = DMA_Priority_High;
41 //不使用 FIFO 模式
42 DMA_InitStructure.DMA_FIFOMode = DMA_FIFOMode_Disable;
43 DMA_InitStructure.DMA_FIFOThreshold = DMA_FIFOThreshold_1QuarterFull;
44 //外设突发单次传输
45 DMA_InitStructure.DMA_MemoryBurst = DMA_MemoryBurst_Single;
46 //存储器突发单次传输
47 DMA_InitStructure.DMA_PeripheralBurst = DMA_PeripheralBurst_Single;
48 DMA_Init(I2Sx_TX_DMA_STREAM, &DMA_InitStructure); //初始化 DMA Stream
49
50 //双缓冲模式配置
51 DMA_DoubleBufferModeConfig(I2Sx_TX_DMA_STREAM, (uint32_t)buffer0, DMA_Memory_0);
52
53 DMA_DoubleBufferModeConfig(I2Sx_TX_DMA_STREAM, (uint32_t)buffer1, DMA_Memory_1);
54
55 //双缓冲模式开启
56 DMA_DoubleBufferModeCmd(I2Sx_TX_DMA_STREAM, ENABLE);
57
58 //开启传输完成中断
59 DMA_ITConfig(I2Sx_TX_DMA_STREAM, DMA_IT_TC, ENABLE);
60 //SPI2 TX DMA 请求使能.
61 SPI_I2S_DMACmd(WM8978_I2Sx_SPI, SPI_I2S_DMAReq_Tx, ENABLE);
62
63 NVIC_InitStructure.NVIC_IRQChannel = I2Sx_TX_DMA_STREAM_IRQn;
64 NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = 0;
65 NVIC_InitStructure.NVIC_IRQChannelSubPriority = 1;
66 NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;
67 NVIC_Init(&NVIC_InitStructure);
68 }
```

I2Sx_TX_DMA_Init 函数用于初始化 I2S 数据发送 DMA 请求工作环境，并启动 DMA 传输。它有三个形参，第一个为缓冲区 1 地址，第二个为缓冲区 2 地址，第三为缓冲区大小。这里使用 DMA 的双缓冲区模式，就是开辟两个缓冲区空间，当第一个缓冲区用于 DMA 传输时(不占用 CPU)，CPU 可以往第二个缓冲区填充数据，等到第一个缓冲区 DMA 传输完成后切换第二个缓冲区用于 DMA 传输，CPU 往第一个缓冲区填充数据，如此不断循环切换，可以达到 DMA 数据传输不间断效果，具体可参考 DMA 章节。这里为保证播放流畅性使用了 DMA 双缓冲区模式。

I2Sx_TX_DMA_Init 函数首先是使能 I2S 发送 DMA 流时钟，并复位 DMA 流配置和相关中断标志位。

通过对 DMA_InitTypeDef 结构体类型的变量赋值配置 DMA 流工作环境并通过 DMA_Init 完成配置。

DMA_DoubleBufferModeConfig 函数用于指定 DMA 双缓冲区模式下缓冲区地址。通过 DMA_DoubleBufferModeCmd 函数可以开启 DMA 双缓冲区模式。这里使能 DMA 传输完成中, 用于指示其中一个缓冲区传输完成, 需要切换缓冲区, 可以开始往缓冲区填充数据。

SPI_I2S_DMAMCmd 用于使能 I2S 发送 DMA 传输请求。

最后配置 DMA 传输完成中断的优先级。

DMA 数据发送传输完成中断服务函数

代码清单 42-13 DMA 数据发送传输完成中断服务函数

```
1 //I2S DMA 回调函数
2 void (*I2S_DMA_TX_Callback)(void);
3
4 void I2Sx_TX_DMA_STREAM_IRQFUN(void)
5 {
6     //DMA 传输完成标志
7     if (DMA_GetITStatus(I2Sx_TX_DMA_STREAM, I2Sx_TX_DMA_IT_TCIF) == SET) {
8         //清 DMA 传输完成标准
9         DMA_ClearITPendingBit(I2Sx_TX_DMA_STREAM, I2Sx_TX_DMA_IT_TCIF);
10        //执行回调函数, 读取数据等操作在这里面处理
11        I2S_DMA_TX_Callback();
12    }
13 }
```

首先声明一个函数指针, 即 I2S_DMA_TX_Callback 是一个函数指针, 一般是先定义一个函数, 再把函数名称赋值给 I2S_DMA_TX_Callback。

I2Sx_TX_DMA_STREAM_IRQFUN 函数是 I2S 的 DMA 传输中断服务函数, 在判断是 DMA 传输完成中断后执行 I2S_DMA_TX_Callback 函数指针对应函数内容。

启动和停止播放控制

代码清单 42-14 启动和停止播放控制

```
1 void I2S_Play_Start(void)
2 {
3     DMA_Cmd(I2Sx_TX_DMA_STREAM, ENABLE); //开启 DMA TX 传输, 开始播放
4 }
5
6
7 void I2S_Play_Stop(void)
8 {
9     DMA_Cmd(I2Sx_TX_DMA_STREAM, DISABLE); //关闭 DMA TX 传输, 结束播放
10 }
11
```

I2S_Play_Start 用于开始播放, I2S_Play_Stop 用于停止播放, 实际是通过控制 DMA 传输使能来实现。

I2S 扩展功能模式配置

代码清单 42-15 I2Sxext_Mode_Config 函数

```
1 void I2Sxext_Mode_Config(const uint16_t _usStandard,
2                           const uint16_t _usWordLen, const uint32_t _usAudioFreq)
3 {
4     I2S_InitTypeDef I2Sext_InitStructure;
5
6     /* I2S2 外设置 */
7     /* 配置 I2S 工作模式 */
8     I2Sext_InitStructure.I2S_Mode = I2S_Mode_MasterTx;
9     /* 接口标准 */
10    I2Sext_InitStructure.I2S_Standard = _usStandard;
```

```
11  /* 数据格式, 16bit */
12  I2Sext_InitStructure.I2S_DataFormat = _usWordLen;
13  /* 主时钟模式 */
14  I2Sext_InitStructure.I2S_MCLKOutput = I2S_MCLKOutput_Enable;
15  /* 音频采样频率 */
16  I2Sext_InitStructure.I2S_AudioFreq = _usAudioFreq;
17  I2Sext_InitStructure.I2S_CPOL = I2S_CPOL_Low;
18
19  I2S_FullDuplexConfig(WM8978_I2Sx_ext, &I2Sext_InitStructure);
20
21  /* 使能 SPI2/I2S2 外设 */
22  I2S_Cmd(WM8978_I2Sx_ext, ENABLE);
23 }
```

I2Sxext_Mode_Config 函数用于配置 I2S 全双工模式, 使用扩展 I2S 功能方便数据处理, 这对实现录音功能有很大的帮助。它有三个形参, 第一个为指定 I2S 接口标准, 一般设置为 I2S Philips 标准, 第二个为字长设置, 一般设置为 16bit, 第三个为采样频率, 一般设置为 44KHz 既可得到高音质效果。

I2Sxext_Mode_Config 函数没有配置 I2S 相关时钟, 一般在 I2Sx_Mode_Config 函数完成时钟配置, 所以一般先运行 I2Sx_Mode_Config 函数才运行 I2Sxext_Mode_Config 函数。

I2Sxext_Mode_Config 函数先给 I2S_InitTypeDef 结构体类型的变量赋值配置参数, 然后调用 I2S_FullDuplexConfig 函数配置 I2S 全双工模式。最后使用 I2S_Cmd 使能扩展 I2S。

I2S 扩展数据接收(DMA 传输)

代码清单 42-16 I2Sxext_RX_DMA_Init 函数

```
1 void I2Sxext_RX_DMA_Init(const uint16_t *buffer0, const uint16_t *buffer1,
2                           const uint32_t num)
3 {
4     NVIC_InitTypeDef  NVIC_InitStructure;
5     DMA_InitTypeDef   DMA_InitStructure;
6
7     RCC_AHB1PeriphClockCmd(I2Sx_DMA_CLK, ENABLE);
8
9     DMA_DeInit(I2Sxext_RX_DMA_STREAM);
10    while (DMA_GetCmdStatus(I2Sxext_RX_DMA_STREAM) != DISABLE) {}
11
12    DMA_ClearITPendingBit(I2Sxext_RX_DMA_STREAM,
13        DMA_IT_FEIF3|DMA_IT_DMEIF3|DMA_IT_TEIF3|DMA_IT_HTIF3|DMA_IT_TCIF3);
14
15    /* 配置 DMA Stream */
16    DMA_InitStructure.DMA_Channel = I2Sxext_RX_DMA_CHANNEL;
17    DMA_InitStructure.DMA_PeripheralBaseAddr = (uint32_t)&WM8978_I2Sx_ext->DR;
18
19    DMA_InitStructure.DMA_Memory0BaseAddr = (uint32_t)buffer0;
20    DMA_InitStructure.DMA_DIR = DMA_DIR_PeripheralToMemory;
21    DMA_InitStructure.DMA_BufferSize = num;
22    DMA_InitStructure.DMA_PeripheralInc = DMA_PeripheralInc_Disable;
23    DMA_InitStructure.DMA_MemoryInc = DMA_MemoryInc_Enable;
24    DMA_InitStructure.DMA_PeripheralDataSize = DMA_PeripheralDataSize_HalfWord;
25
26    DMA_InitStructure.DMA_MemoryDataSize = DMA_MemoryDataSize_HalfWord;
27    DMA_InitStructure.DMA_Mode = DMA_Mode_Circular;
28    DMA_InitStructure.DMA_Priority = DMA_Priority_Medium;
29    DMA_InitStructure.DMA_FIFOMode = DMA_FIFOMode_Disable;
30    DMA_InitStructure.DMA_FIFOThreshold = DMA_FIFOThreshold_1QuarterFull;
31    DMA_InitStructure.DMA_MemoryBurst = DMA_MemoryBurst_Single;
32    DMA_InitStructure.DMA_PeripheralBurst = DMA_PeripheralBurst_Single;
33    DMA_Init(I2Sxext_RX_DMA_STREAM, &DMA_InitStructure);
34
35    DMA_DoubleBufferModeConfig(I2Sxext_RX_DMA_STREAM, (uint32_t)buffer0, DMA_Memory_0);
36}
```



```
37 DMA_DoubleBufferModeConfig(I2Sxext_RX_DMA_STREAM, (uint32_t)buffer1,DMA_Memory_1);
38
39
40 DMA_DoubleBufferModeCmd(I2Sxext_RX_DMA_STREAM,ENABLE);
41
42 DMA_ITConfig(I2Sxext_RX_DMA_STREAM,DMA_IT_TC,ENABLE);
43
44 SPI_I2S_DMACmd(WM8978_I2Sx_ext,SPI_I2S_DMAReq_Rx,ENABLE);
45
46 NVIC_InitStructure.NVIC_IRQChannel = I2Sxext_RX_DMA_STREAM_IRQn;
47 NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = 0;
48 NVIC_InitStructure.NVIC_IRQChannelSubPriority = 2;
49 NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;
50 NVIC_Init(&NVIC_InitStructure);
51 }
```

I2Sxext_RX_DMA_Init 函数配置扩展 I2S 的数据接收功能，使用 DMA 传输方式接收数据，程序结构与 I2Sx_TX_DMA_Init 函数一致，只是 DMA 传输方向不同，I2Sxext_RX_DMA_Init 函数是从外设到存储器传输，I2Sx_TX_DMA_Init 函数是存储器到外设传输。

I2Sxext_RX_DMA_Init 函数也是使用 DMA 的双缓冲区模式传输数据。最后使能了 DMA 传输完成中断，并使能 DMA 数据接收请求。

DMA 数据接收传输完成中断服务函数

代码清单 42-17 DMA 数据接收传输完成中断服务函数

```
1 //I2S DMA RX 回调函数
2 void (*I2S_DMA_RX_Callback)(void);
3
4 void I2Sxext_RX_DMA_STREAM_IRQFUN(void)
5 {
6     if(DMA_GetITStatus(I2Sxext_RX_DMA_STREAM,I2Sxext_RX_DMA_IT_TCIF)==SET) {
7
8         DMA_ClearITPendingBit(I2Sxext_RX_DMA_STREAM,I2Sxext_RX_DMA_IT_TCIF);
9         I2S_DMA_RX_Callback(); //执行回调函数,读取数据等操作在这里面处理
10    }
```

与 DMA 数据发送传输完成中断服务函数类似，I2Sxext_RX_DMA_STREAM_IRQFUN 函数在判断得到是数据接收传输完成后执行 I2S_DMA_RX_Callback 函数，I2S_DMA_RX_Callback 实际也是一个函数指针。

启动和停止录音

代码清单 42-18 启动和停止录音

```
1 void I2Sxext_Record_Start(void)
2 {
3     DMA_Cmd(I2Sxext_RX_DMA_STREAM,ENABLE);
4 }
5
6 void I2Sxext_Record_Stop(void)
7 {
8     DMA_Cmd(I2Sxext_RX_DMA_STREAM,DISABLE);
9 }
10
```

I2Sxext_Record_Start 函数用于启动录音，I2Sxext_Record_Stop 函数用于停止录音，实际是通过控制 DMA 传输使能来实现。

至此，关于 WM8978 芯片的驱动程序已经介绍完整了，该部分程序都是在 bsp_wm8978.c 文件中的，接下来我们就可以使用这些驱动程序实现录音和回放功能了。

3. 录音和回放功能

录音和回放功能是在 WM8978 驱动函数基础上搭建而成的，实现代码存放在 Recorder.c 和 Recorder.h 文件中。启动录音功能后会在 SD 卡内创建一个 WAV 格式文件，把音频数据保存在该文件中，录音结束后既可得到一个完整的 WAV 格式文件。回放功能用于播放录音文件，实际上回放功能的实现函数也是适用于播放其他 WAV 格式文件的。

枚举和结构体类型定义

代码清单 42-19 枚举和结构体类型定义

```
1  /* 录音机状态 */
2  enum {
3      STA_IDLE = 0, /* 待机状态 */
4      STA_RECORDING, /* 录音状态 */
5      STA_PLAYING, /* 放音状态 */
6      STA_ERR, /* error */
7  };
8
9  typedef struct {
10     uint8_t ucInput; /* 输入源: 0:MIC, 1:线输入 */
11     uint8_t ucFmtIdx; /* 音频格式: 标准、位长、采样频率 */
12     uint8_t ucVolume; /* 当前放音音量 */
13     uint8_t ucGain; /* 当前增益 */
14     uint8_t ucStatus; /* 录音机状态, 0 表示待机, 1 表示录音中, 2 表示播放中 */
15 } REC_TYPE;
16
17 /* WAV 文件头格式 */
18 typedef __packed struct {
19     uint32_t riff; /* = "RIFF" 0x46464952 */
20     uint32_t size_8; /* 从下个地址开始到文件尾的总字节数 */
21     uint32_t wave; /* = "WAVE" 0x45564157 */
22
23     uint32_t fmt; /* = "fmt " 0x20746d66 */
24     uint32_t fmtSize; /* 下一个结构体的大小(一般为 16) */
25     uint16_t wFormatTag; /* 编码方式, 一般为 1 */
26     uint16_t wChannels; /* 通道数, 单声道为 1, 立体声为 2 */
27     uint32_t dwSamplesPerSec; /* 采样率 */
28     uint32_t dwAvgBytesPerSec; /* 每秒字节数(= 采样率 × 每个采样点字节数) */
29     uint16_t wBlockAlign; /* 每个采样点字节数(= 量化比特数/8*通道数) */
30     uint16_t wBitsPerSample; /* 量化比特数(每个采样需要的 bit 数) */
31
32     uint32_t data; /* = "data" 0x61746164 */
33     uint32_t datasize; /* 纯数据长度 */
34 } WavHead;
```

首先，定义一个枚举类型罗列录音和回放功能的状态，录音和回放功能是不能同时使用的，使用枚举类型区分非常有效。

REC_TYPE 结构体类型定义了录音和回放功能相关可控参数，包括 WM8978 声音源输入端，可选板载咪头或板载耳机插座的 LINE 线输入；音频格式选择，一般选择 I2S Philips 标准、16bit 字长、44KHz 采样频率；音频输出耳机音量控制；录音时声音增益；当前状态。

WavHead 结构体类型定义了 WAV 格式文件头，具体参考“WAV 格式文件”，这里没有用到 fact chunk。需要注意的是这里使用 __packed 关键字，它表示结构字节对齐。

启动播放 WAV 格式音频文件

代码清单 42-20 StartPlay 函数

```
1 static void StartPlay(const char *filename)
2 {
3     printf("当前播放文件 -> %s\n", filename);
4
5     result=f_open(&file, filename, FA_READ);
6     if (result!=FR_OK) {
7         printf("打开音频文件失败!!!->%d\r\n", result);
8         result = f_close (&file);
9         Recorder.ucStatus = STA_ERR;
10        return;
11    }
12    //读取 WAV 文件头
13    result = f_read(&file, &rec_wav, sizeof(rec_wav), &bw);
14    //先读取音频数据到缓冲区
15    result = f_read(&file, (uint16_t *)buffer0, RECBUFFER_SIZE*2, &bw);
16    result = f_read(&file, (uint16_t *)buffer1, RECBUFFER_SIZE*2, &bw);
17
18    Delay_ms(10); /* 延迟一段时间, 等待 I2S 中断结束 */
19    I2S_Stop(); /* 停止 I2S 录音和放音 */
20    wm8978_Reset(); /* 复位 WM8978 到复位状态 */
21
22    Recorder.ucStatus = STA_PLAYING; /* 放音状态 */
23
24    /* 配置 WM8978 芯片, 输入为 DAC, 输出为耳机 */
25    wm8978_CfgAudioPath(DAC_ON, EAR_LEFT_ON | EAR_RIGHT_ON);
26    /* 调节音量, 左右相同音量 */
27    wm8978_SetOUT1Volume(Recorder.ucVolume);
28    /* 配置 WM8978 音频接口为飞利浦标准 I2S 接口, 16bit */
29    wm8978_CfgAudioIF(I2S_Standard_Phillips, 16);
30
31    I2Sx_Mode_Config(g_FmtList[Recorder.ucFmtIdx][0],
32        g_FmtList[Recorder.ucFmtIdx][1], g_FmtList[Recorder.ucFmtIdx][2]);
33    I2Sxext_Mode_Config(g_FmtList[Recorder.ucFmtIdx][0],
34        g_FmtList[Recorder.ucFmtIdx][1], g_FmtList[Recorder.ucFmtIdx][2]);
35
36    I2Sxext_RX_DMA_Init(&recplaybuf[0], &recplaybuf[1], 1);
37    DMA_ITConfig(I2Sxext_RX_DMA_STREAM, DMA_IT_TC, DISABLE); //开启传输完成中断
38    I2Sxext_Record_Stop();
39
40    I2Sx_TX_DMA_Init(buffer0, buffer1, RECBUFFER_SIZE);
41    I2S_Play_Start();
42 }
```

StartPlay 函数用于启动播放 WAV 格式音频文件, 它有一个形参, 用于指示待播放文件名称。函数首先检查待播放文件是否可以正常打开, 如果打开失败则退出播放。如果可以正常打开文件则先读取 WAV 格式文件头, 保存在 WavHead 结构体类型变量 rec_wav 中, 同时先读取音频数据填充到两个缓冲区中, 这两个缓冲区 buffer0 和 buffer1 是定义的全局数组变量, 用于 DMA 双缓冲区模式。

接下来, 配置 WM8978 的工作环境, 首先停止 I2S 并复位 WM8978 芯片。这里是播放音频功能, 所以设置 WM8978 的输入是 DAC, 播放 I2S 接口接收到的音频数据, 输出设置为耳机输出。STM32 控制器的 I2S 接口和 WM8978 的 I2S 接口都设置为 I2S Philips 标志、字长为 16bit。

然后, 调用 I2Sx_TX_DMA_Init 配置 I2S 的 DMA 发送请求, 并调用 I2S_Play_Start 函数使能 DMA 数据传输。

启动录音功能

代码清单 42-21 StartRecord 函数

```
1 static void StartRecord(const char *filename)
2 {
3     printf("当前录音文件 -> %s\n", filename);
4     result=f_open(&file, filename, FA_CREATE_ALWAYS|FA_WRITE);
5     if (result!=FR_OK) {
6         printf("Open wavfile fail!!!->%d\r\n", result);
7         result = f_close (&file);
8         Recorder.ucStatus = STA_ERR;
9         return;
10    }
11
12    // 写入 WAV 文件头, 这里必须写入写入后文件指针自动偏移到 sizeof(rec_wav) 位置,
13    // 接下来写入音频数据才符合格式要求。
14    result=f_write(&file, (const void *)&rec_wav, sizeof(rec_wav), &bw);
15
16    Delay_ms(10);    /* 延迟一段时间, 等待 I2S 中断结束 */
17    I2S_Stop();    /* 停止 I2S 录音和放音 */
18    wm8978_Reset();    /* 复位 WM8978 到复位状态 */
19
20    Recorder.ucStatus = STA_RECORDING;    /* 录音状态 */
21
22    /* 调节放音音量, 左右相同音量 */
23    wm8978_SetOUT1Volume(Recorder.ucVolume);
24
25    if (Recorder.ucInput == 1) { /* 线输入 */
26        /* 配置 WM8978 芯片, 输入为线输入, 输出为耳机 */
27        wm8978_CfgAudioPath(LINE_ON | ADC_ON, EAR_LEFT_ON | EAR_RIGHT_ON);
28        wm8978_SetLineGain(Recorder.ucGain);
29    } else { /* MIC 输入 */
30        /* 配置 WM8978 芯片, 输入为 Mic, 输出为耳机 */
31        wm8978_CfgAudioPath(MIC_RIGHT_ON|ADC_ON, EAR_LEFT_ON|EAR_RIGHT_ON);
32        wm8978_SetMicGain(Recorder.ucGain);
33    }
34
35    /* 配置 WM8978 音频接口为飞利浦标准 I2S 接口, 16bit */
36    wm8978_CfgAudioIF(I2S_Standard_Phillips, 16);
37
38    I2Sx_Mode_Config(g_FmtList[Recorder.ucFmtIdx][0],
39        g_FmtList[Recorder.ucFmtIdx][1], g_FmtList[Recorder.ucFmtIdx][2]);
40    I2Sxext_Mode_Config(g_FmtList[Recorder.ucFmtIdx][0],
41        g_FmtList[Recorder.ucFmtIdx][1], g_FmtList[Recorder.ucFmtIdx][2]);
42
43    I2Sx_TX_DMA_Init(&recplaybuf[0], &recplaybuf[1], 1);
44    DMA_ITConfig(I2Sx_TX_DMA_STREAM, DMA_IT_TC, DISABLE); //开启传输完成中断
45
46    I2S_DMA_RX_Callback=Recorder_I2S_DMA_RX_Callback;
47    I2Sxext_RX_DMA_Init(buffer0, buffer1, RECBUFFER_SIZE);
48
49    I2S_Play_Start();
50    I2Sxext_Record_Start();
51 }
```

StartRecord 函数在结构上与 StartPlay 函数类似, 它实现启动录音功能, 它有一个形参, 指示保存录音数据的文件名称。StartRecord 函数会首先创建录音文件, 因为用到 FA_CREATE_ALWAYS 标志位, f_open 函数会总是创建新文件, 如果已存在该文件则会覆盖原先的文件内容。

这些必须先写入 WAV 格式文件头数据, 这样当前文件指针自动移动到文件头的下一个字节, 即是存放音频数据的起始位置。

开发板支持 LINE 线输入和板载咪头输入, 程序默认使用咪头输入, 在录音同时使能耳机输出, 这样录音时在耳机接口是有相同的声音输出的。

录音功能需要使能扩展 I2S。

录音和回放功能选择

代码清单 42-22 RecorderDemo 函数

```
1 void RecorderDemo(void)
2 {
3     uint8_t i;
4     uint8_t ucRefresh; /* 通过串口打印相关信息标志 */
5     DIR dir;
6
7     Recorder.ucStatus=STA_IDLE; /* 开始设置为空闲状态 */
8     Recorder.ucInput=0; /* 缺省 MIC 输入 */
9     Recorder.ucFmtIdx=3; /* 缺省飞利浦 I2S 标准, 16bit 数据长度, 44K 采样率 */
10    Recorder.ucVolume=35; /* 缺省耳机音量 */
11    if (Recorder.ucInput==0) { //MIC
12        Recorder.ucGain=50; /* 缺省 MIC 增益 */
13        rec_wav.wChannels=1; /* 缺省 MIC 单通道 */
14    } else { //LINE
15        Recorder.ucGain=6; /* 缺省线路输入增益 */
16        rec_wav.wChannels=2; /* 缺省线路输入双声道 */
17    }
18
19    rec_wav.riff=0x46464952; /* "RIFF"; RIFF 标志 */
20    rec_wav.size_8=0; /* 文件长度, 未确定 */
21    rec_wav.wave=0x45564157; /* "WAVE"; WAVE 标志 */
22
23    rec_wav.fmt=0x20746d66; /* "fmt "; fmt 标志, 最后一位为空 */
24    rec_wav.fmtSize=16; /* sizeof(PCMWAVEFORMAT) */
25    rec_wav.wFormatTag=1; /* 1 表示为 PCM 形式的声音数据 */
26    /* 每样本的数据位数, 表示每个声道中各个样本的数据位数。 */
27    rec_wav.wBitsPerSample=16;
28    /* 采样频率 (每秒样本数) */
29    rec_wav.dwSamplesPerSec=g_FmtList[Recorder.ucFmtIdx][2];
30    /* 每秒数据量; 其值为通道数×每秒数据位数×每样本的数据位数 / 8。 */
31    rec_wav.dwAvgBytesPerSec=
32        rec_wav.wChannels*rec_wav.dwSamplesPerSec*rec_wav.wBitsPerSample/8;
33    /* 数据块的调整数 (按字节算的), 其值为通道数×每样本的数据位数 / 8。 */
34    rec_wav.wBlockAlign=rec_wav.wChannels*rec_wav.wBitsPerSample/8;
35
36    rec_wav.data=0x61746164; /* "data"; 数据标记符 */
37    rec_wav.datasize=0; /* 语音数据大小 目前未确定 */
38
39    /* 如果路径不存在, 创建文件夹 */
40    result = f_opendir(&dir, RECORDERDIR);
41    while (result != FR_OK) {
42        f_mkdir(RECORDERDIR);
43        result = f_opendir(&dir, RECORDERDIR);
44    }
45
46    /* 初始化并配置 I2S */
47    I2S_Stop();
48    I2S_GPIO_Config();
49    I2Sx_Mode_Config(g_FmtList[Recorder.ucFmtIdx][0],
50        g_FmtList[Recorder.ucFmtIdx][1], g_FmtList[Recorder.ucFmtIdx][2]);
51    I2Sxext_Mode_Config(g_FmtList[Recorder.ucFmtIdx][0],
52        g_FmtList[Recorder.ucFmtIdx][1], g_FmtList[Recorder.ucFmtIdx][2]);
53
54    I2S_DMA_TX_Callback=MusicPlayer_I2S_DMA_TX_Callback;
55    I2S_Play_Stop();
56
57    I2S_DMA_RX_Callback=Recorder_I2S_DMA_RX_Callback;
58    I2Sxext_Record_Stop();
59
```

```
60     ucRefresh = 1;
61     bufflag=0;
62     Isread=0;
63     /* 进入主程序循环体 */
64     while (1) {
65         /* 如果使能串口打印标志则打印相关信息 */
66         if (ucRefresh == 1) {
67             DispStatus(); /* 显示当前状态, 频率, 音量等 */
68             ucRefresh = 0;
69         }
70         if (Recorder.ucStatus == STA_IDLE) {
71             /* KEY2 开始录音 */
72             if (Key_Scan(KEY2_GPIO_PORT, KEY2_PIN) == KEY_ON) {
73                 /* 寻找合适文件名 */
74                 for (i=1; i<0xff; ++i) {
75                     sprintf(recfilename, "0:/recorder/rec%03d.wav", i);
76                     result=f_open(&file, (const TCHAR *)recfilename, FA_READ);
77                     if (result==FR_NO_FILE) break;
78                 }
79                 f_close(&file);
80
81                 if (i==0xff) {
82                     Recorder.ucStatus = STA_ERR;
83                     continue;
84                 }
85                 /* 开始录音 */
86                 StartRecord(recfilename);
87                 ucRefresh = 1;
88             }
89             /* TouchPAD 开始回放录音 */
90             if (TPAD_Scan(0)) {
91                 /* 开始回放 */
92                 StartPlay(recfilename);
93                 ucRefresh = 1;
94             }
95         } else {
96             /* KEY1 停止录音或回放 */
97             if (Key_Scan(KEY1_GPIO_PORT, KEY1_PIN) == KEY_ON) {
98                 /* 对于录音, 需要把 WAV 文件内容填充完整 */
99                 if (Recorder.ucStatus == STA_RECORDING) {
100                     I2Sxext_Record_Stop();
101                     I2S_Play_Stop();
102                     rec_wav.size_8=wavsize+36;
103                     rec_wav.datasize=wavsize;
104                     result=f_lseek(&file, 0);
105                     result=f_write(&file, (const void *)&rec_wav,
106                                   sizeof(rec_wav), &bw);
107                     result=f_close(&file);
108                     printf("录音结束\r\n");
109                 }
110                 ucRefresh = 1;
111                 Recorder.ucStatus = STA_IDLE; /* 待机状态 */
112                 I2S_Stop(); /* 停止 I2S 录音和放音 */
113                 wm8978_Reset(); /* 复位 WM8978 到复位状态 */
114             }
115         }
116         /* DMA 传输完成 */
117         if (Isread==1) {
118             Isread=0;
119             switch (Recorder.ucStatus) {
120                 case STA_RECORDING: // 录音功能, 写入数据到文件
121                     if (bufflag==0)
122                         result=f_write(&file, buffer0, RECBUFFER_SIZE*2, (UINT*)&bw);
123                     else
124                         result=f_write(&file, buffer1, RECBUFFER_SIZE*2, (UINT*)&bw);
```



```
125         wavsize+=RECBUFFER_SIZE*2;
126         break;
127     case STA_PLAYING:    // 回放功能，读取数据到播放缓冲区
128         if (bufflag==0)
129             result = f_read(&file,buffer0,RECBUFFER_SIZE*2,&bw);
130         else
131             result = f_read(&file,buffer1,RECBUFFER_SIZE*2,&bw);
132         /* 播放完成或读取出错停止工作 */
133         if ((result!=FR_OK)|| (file.fptr==file.fsize)) {
134             printf("播放完或者读取出错退出...\r\n");
135             I2S_Play_Stop();
136             file.fptr=0;
137             f_close(&file);
138             Recorder.ucStatus = STA_IDLE;    /* 待机状态 */
139             I2S_Stop();    /* 停止 I2S 录音和放音 */
140             wm8978_Reset(); /* 复位 WM8978 到复位状态 */
141         }
142         break;
143     }
144 }
145 }
146 }
```

RecorderDemo 函数实现录音和回放功能。Recorder 是一个 REC_TYPE 结构体类型变量，指示录音和回放功能相关参数，这里通过赋值缺省选择板载咪头输入、使用 I2S Philips 标准、16bit 字长、44KHz 采样频率，并设置了音量和增益，对于 LINE 输入增益范围为 0~7。rec_wav 是 WavHead 结构体类型变量，用于设置 WAV 格式文件头，很多成员赋值为缺省值即可，成员 size_8 和 datasize 变量表示数据大小，因为录音时间长度直接影响这两个变量大小，现在并无法确定它们大小，需要在停止录音后才可计算得到它们的值。

接下来是使用 FatFs 的功能函数 f_opendir 和 f_mkdir 组合判断 SD 卡内是否有名为“recorder”的文件夹，如果没有改文件夹就创建它，因为我们打算把录音文件存放在该文件夹内。

接下来就是调用 I2S 相关函数完成 I2S 工作环境配置。

MusicPlayer_I2S_DMA_TX_Callback 是我们定义的一个函数的函数名，把他赋值给函数指针 I2S_DMA_TX_Callback，这样可以实现在执行 DMA 数据发送完成中断服务函数时执行 MusicPlayer_I2S_DMA_TX_Callback 函数。Recorder_I2S_DMA_RX_Callback 情况与之类似。开始时停止录音和回放功能。

ucRefresh 变量作为通过串口打印相关操作和状态信息到串口调试助手“刷新”标志。bufflag 变量用于指示当前空闲缓冲区，工程定义了两个缓冲区 buffer0 和 buffer1 用于 DMA 双缓冲区模式，对于录音功能，buffag 为 0 表示当前 DMA 使用 buffer1 填充，buffer0 已经填充完整，为 1 表示当前 DMA 使用 buffer0 填充，buffer1 已经填充完整；对于回放功能，buffag 为 0 表示 buffer1 用于当前播放，buffer0 已经播放完成需要读取新数据填充，为 1 表示 buffer0 用于当前播放，buffer1 已经播放完成需要读取新数据填充。Isread 变量用于指示 DMA 传输完成状态，为 1 时表示 DMA 传输完成，只有在 DMA 传输完成中断服务函数中才会被置 1。

接下来就是无限循环函数了，先判断 ucRefresh 变量状态，如果为 1 就执行 DispStatus 函数，该函数只是使用 printf 函数打印相关信息。开发板集成有两个独立按键 K1 和 K2，

还有一个电容按键，程序设置按下 K2 按键开始录音功能，触摸电容按键开始回放功能，按下 K1 按键用于停止录音和回放功能，这里 K2 按键和电容按键是互斥的。

在空闲状态下，允许按下 K2 按键启动录音，录音功能是通过调用 StartRecord 函数启动的，该函数需要一个参数指示录音文件名称，使用在执行 StartRecord 函数之前会循环使用 f_open 函数获取 SD 卡内不存在的文件，防止覆盖已存在文件。

在空闲状态下，允许触摸电容按键启动回放功能，它直接使用 StartPlay 函数启动上一个录音文件播放。

不在空闲状态下，按下 K1 按键可以停在录音或回放功能，回放功能只需要停在 I2S 的 DMA 数据发送接口并复位 WM8978 即可，录音功能需要停在扩展 I2S 的 DMA 数据接收功能，并且需要填充完整 WAV 格式文件头并写入到录音文件中。

无限循环还要检测 Isread 变量的状态，当它在 DMA 传输完成中断服务函数被置 1 时说明双缓冲区状态发生改变，对于录音功能，需要把以填满的缓冲区数据通过 f_write 写入到文件中；对于回放功能，需要利用 f_read 函数读取新数据填充到已播缓冲区中，如果遇到读取出错或文件已经播放完全需要停止 I2S 的 DMA 传输并复位 WM8978。

DMA 传输完成中断回调函数

代码清单 42-23 DMA 传输完成中断回调函数

```
1 /* DMA 发送完成中断回调函数 */
2 /* 缓冲区内容已经播放完成，需要切换缓冲区，进行新缓冲区内容播放
3    同时读取 WAV 文件数据填充到已播缓冲区 */
4 void MusicPlayer_I2S_DMA_TX_Callback(void)
5 {
6     if (Recorder.ucStatus == STA_PLAYING) {
7         if (I2Sx_TX_DMA_STREAM->CR&(1<<19)) { //当前使用 Memory1 数据
8             bufflag=0;                          //可以将数据读取到缓冲区 0
9         } else {                                //当前使用 Memory0 数据
10            bufflag=1;                          //可以将数据读取到缓冲区 1
11        }
12        Isread=1;                               // DMA 传输完成标志
13    }
14 }
15
16 /* DMA 接收完成中断回调函数 */
17 /* 录音数据已经填满了一个缓冲区，需要切换缓冲区，
18    同时可以把已满的缓冲区内容写入到文件中 */
19 void Recorder_I2S_DMA_RX_Callback(void)
20 {
21     if (Recorder.ucStatus == STA_RECORDING) {
22         if (I2Sxext_RX_DMA_STREAM->CR&(1<<19)) { //当前使用 Memory1 数据
23             bufflag=0;
24         } else {                                //当前使用 Memory0 数据
25             bufflag=1;
26         }
27        Isread=1;                               // DMA 传输完成标志
28    }
29 }
```

这两个函数用于在 DMA 传输完成后切换缓冲区。DMA 数据流 x 配置寄存器的 CT 位用于指示当前目标缓冲区，如果为 1，当前目标缓冲区为存储器 1；如果为 0，则为存储器 0。

主函数

代码清单 42-24 main 函数

```
1 int main(void)
2 {
3     FRESULT result;
4
5     /* NVIC 中断优先级组选择 */
6     NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2);
7
8     /* 关闭 BL8782wifi 使能 */
9     BL8782_PDN_INIT();
10
11    /* 初始化按键 */
12    Key_GPIO_Config();
13
14    /* 初始化调试串口，一般为串口 1 */
15    Debug_USART_Config();
16
17    /* 挂载 SD 卡文件系统 */
18    result = f_mount(&fs, "0:", 1); //挂载文件系统
19    if (result != FR_OK) {
20        printf("\n SD 卡文件系统挂载失败\n");
21        while (1);
22    }
23
24    /* 初始化系统滴答定时器 */
25    SysTick_Init();
26    printf("WM8978 录音和回放功能\n");
27
28    /* 初始化电容按键 */
29    TPAD_Init();
30
31    /* 检测 WM8978 芯片，此函数会自动配置 CPU 的 GPIO */
32    if (wm8978_Init() == 0) {
33        printf("检测不到 WM8978 芯片!!!\n");
34        while (1); /* 停机 */
35    }
36    printf("初始化 WM8978 成功\n");
37
38    /* 录音与回放功能 */
39    RecorderDemo();
40 }
```

main 函数主要完成各个外设的初始化，包括独立按键初始化、电容按键初始化、调试串口初始化、SD 卡文件系统挂载还有系统定时器初始化。

wm8978_Init 初始化 I2C 接口用于控制 WM8978 芯片，并复位 WM8978 芯片，如果初始化成功则运行 RecorderDemo 函数进行录音和回放功能测试。

42.6.3 下载验证

把 Micro SD 卡插入到开发板右侧的卡槽内，使用 USB 线连接开发板上的“USB TO UART”接口到电脑，将耳机插入到开发板上边沿左侧的耳机插座，电脑端配置好串口调试助手参数。编译实验程序并下载到开发板上，程序运行后在串口调试助手可接收到开发板发过来的提示信息，按下开发板左下边沿的 K2 按键，开始执行录音功能测试，不断对着咪头说话，就可以把声音录制下来，按下 K1 按键可以停止录音。然后触摸电容按键就

可以在耳机接口听到之前录音内容了，按下 K1 按键可停止播放。录音完成后也可以在电脑端打开 SD 卡，找到其中的录音文件，可在电脑端音频播放器播放录音文件。

42.7 MP3 播放器

MP3 格式音乐文件普遍存在我们生活中，实际上 MP3 本身是一种音频编码方式，全称为 Moving Picture Experts Group Audio Layer III(MPEG Audio Layer 3)。MPEG 音频文件是 MPEG 标准中的声音部分，根据压缩质量和编码复杂程度划分为三层，即 Layer-1、Layer2、Layer3，且分别对应 MP1、MP2、MP3 这三种声音文件，其中 MP3 压缩率可达到 10:1 至 12:1，可以大大减少文件占用存储空间大小。MPEG 音频编码的层次越高，编码器越复杂，压缩率也越高。MP3 是利用人耳对高频声音信号不敏感的特性，将时域波形信号转换成频域信号，并划分成多个频段，对不同的频段使用不同的压缩率，对高频加大压缩比（甚至忽略信号）对低频信号使用小压缩比，保证信号不失真。这样一来就相当于抛弃人耳基本听不到的高频声音，只保留能听到的低频部分，这样可得到很高的压缩率。

42.7.1 MP3 文件结构

MP3 文件大致分为 3 个部分：TAG_V2 (ID3V2)，音频数据，TAG_V1(ID3V1)。ID3 是 MP3 文件中附加关于该 MP3 文件的歌手、标题、专辑名称、年代、风格等等信息，有两个版本 ID3V1 和 ID3V2。ID3V1 固定存放在 MP3 文件末尾，固定长度为 128 字节，以 TAG 三个字符开头，后面跟上歌曲信息。因为 ID3V1 可存储信息量有限，有些 MP3 文件添加了 ID3V2，ID3V2 是可选的，如果存在 ID3V2 那它必然存在在 MP3 文件起始位置，它实际是 ID3V1 的补充。

1. ID3V2

ID3V2 以灵活管理方法 MP3 文件附件信息，比 ID3V1 可以存储更多的信息，同时也比 ID3V1 在结构上复杂得多。常用是 ID3V2.3 版本，一个 MP3 文件最多就一个 ID3V2.3 标签。ID3V2.3 标签由一个标签头和若干个标签帧或一个扩展标签头组成。关于曲目的信息如标题、作者等都存放在不同的标签帧中，扩展标签头和标签帧并不是必要的，但每个标签至少要有一个标签帧。标签头和标签帧一起顺序存放在 MP3 文件的首部。

标签头结构如表 42-6。

表 42-6 标签头

地址	标识	字节	描述
00H	Header	3	字符串“ID3”
03H	Ver	1	版本号，ID3V2.3 就记录为 3
04H	Revision	1	副版本号，一般记录为 0
05H	Flag	1	标志字节，一般不用设置
06H	Size	4	标签大小，整个 ID3V2 所占空间字节的大小

其中标签大小计算如下：

$$\begin{aligned} \text{Total_size} = & (\text{Size}[0] \& 0x7F) * 0x200000 + (\text{Size}[1] \& 0x7F) * 0x400 \\ & + (\text{Size}[2] \& 0x7F) * 0x80 + (\text{Size}[3] \& 0x7F) \end{aligned}$$

每个标签帧都有一个 10 个字节的帧头和至少一个字节的长度不定内容，在文件中是连续存放的。标签帧头由三个部分组成，即 Frame[4]、Size[4]和 Flags[2]。Frame 是用四个字符表示帧内容含义，比如 TIT2 为标题、TPE1 为作者、TALB 为专辑、TRCK 为音轨、TYER 为年代等等信息。Size 用四个字节组成 32bit 数表示帧大小。Flags 是标签帧的标志位，一般为 0 即可。

2. ID3V1

ID3V1 是早期的版本，可以存放的信息量有限，但编程比 ID3V2 简单很多，即使到现在使用还是很多。ID3V1 是固定存放在 MP3 文件末尾的 128 字节，组成结构见表 42-7。

表 42-7 ID3V1 结构

字节	字长	描述
1-3	3	标识符：“TAG”
4-33	30	歌名
34-63	30	作者
64-93	30	专辑名
94-97	4	年份
98-127	30	附注
128	1	MP3 音乐类别

其中，歌名是固定分配为 30 个字节，如果歌名太短则以 0 填充完整，太长则被截断，其他信息类似情况存储。MP3 音乐类别总共有 147 种，每一种对应一个数组，比如 0 对应“Blues”、1 对应“Classic Rock”、2 对应“Country”等等。

3. MP3 数据帧

音频数据是 MP3 文件的主体部分，它由一系列数据帧(Frame)组成，每个 Frame 包含一段音频的压缩数据，通过解码库解码即可得到对应 PCM 音频数据，就可以通过 I2S 发送到 WM8978 芯片播放音乐，按顺序解码所有帧就可以得到整个 MP3 文件的音轨。

每个 Frame 由两部分组成，帧头和数据实体，Frame 长度可能不同，由位率决定。帧头记录了 MP3 数据帧的位率、采样率、版本等等信息，总共有 4 个字节，见表 42-8。

表 42-8 数据帧头结构

名称	位长	描述
Sync	第 1、2 字节	11 同步信息：每位固定为：1。
Version		2 版本：00-MPEG2.5；01-未定义；10-MPEG2；11-MPEG1。
Layer		2 层：00-未定义；01-Layer3；10-Layer2；11-Layer1。
Error Protection		1 CRC 校验：0-校验；1-不校验。
Bitrate index	第 3 字节	4 位率：参考表 42-9。
Sampling frequency		2 采样频率： 对于 MPEG-1：00-44.1kHz；01-48kHz；10-32kHz；11-未定义。 对于 MPEG-2：00-22.05kHz；01-24kHz；10-16kHz；11-未定义。 对于 MPEG-2.5：00-11.025kHz；01-12kHz；10-8kHz；11-未定义。
Padding		1 帧长调整：用来调整文件头长度，0-无需调整；1-调整。
Private		1 保留字。
Mode	第 4 字节	2 声道：00-立体声 Stereo；01-Joint Stereo；10-双声道；11-单声道。
Mode		2 扩充模式：当声道模式为 01 时才使用。

extension	字节		
Copyright		1	版权：文件是否合法，0-不合法；1-合法。
Original		1	原版标志：0-非原版；1-原版。
Emphasis		2	强调方式：用于声音经降噪压缩后再补偿的分类，几乎不用。

位率位在不同版本和层都有不同的定义，具体参考表 42-9，单位为 kbps。其中 V1 对应 MPEG-1，V2 对应 MPEG-2 和 MPEG-2.5；L1 对应 Layer1，L2 对应 Layer2，L3 对应 Layer3，free 表示位率可变，bad 表示该定义不合法。

表 42-9 位率选择

bits	V1, L1	V1, L2	V1, L3	V2, L1	V2, L2	V2, L3
0000	free	free	free	free	free	free
0001	32	32	32	32 (32)	32 (8)	8 (8)
0010	64	48	40	64 (48)	48 (16)	16 (16)
0011	96	56	48	96 (56)	56 (24)	24 (24)
0100	128	64	56	128 (64)	64 (32)	32 (32)
0101	160	80	64	160 (80)	80 (40)	64 (40)
0110	192	96	80	192 (96)	96 (48)	80 (48)
0111	224	112	96	224 (112)	112 (56)	56 (56)
1000	256	128	112	256 (128)	128 (64)	64 (64)
1001	288	160	128	288 (144)	160 (80)	128 (80)
1010	320	192	160	320 (160)	192 (96)	160 (96)
1011	352	224	192	352 (176)	224 (112)	112 (112)
1100	384	256	224	384 (192)	256 (128)	128 (128)
1101	416	320	256	416 (224)	320 (144)	256 (144)
1110	448	384	320	448 (256)	384 (160)	320 (160)
1111	bad	bad	bad	bad	bad	bad

例如，当 Version=11，Layer=01，Bitrate_incdex=0101 时，描述为 MPEG-1

Layer3(MP3)位率为 64kbps。

数据帧长度取决于位率(Bitrate)和采样频率(Sampling_freq)，具体计算如下：

对于 MPEG-1 标准，Layer1 时：

$$\text{Size} = (48000 * \text{Bitrate}) / \text{Sampling_freq} + \text{Padding};$$

Layer2 或 Layer3 时：

$$\text{Size} = (144000 * \text{Bitrate}) / \text{Sampling_freq} + \text{Padding};$$

对于 MPEG-2 或 MPEG-2.5 标准，Layer1 时：

$$\text{Size} = (24000 * \text{Bitrate}) / \text{Sampling_freq} + \text{Padding};$$

Layer2 或 Layer3 时：

$$\text{Size} = (72000 * \text{Bitrate}) / \text{Sampling_freq} + \text{Padding};$$

如果有 CRC 校验，则存放在在帧头后两个字节。接下来就是帧主数据(MAIN_DATA)。一般来说一个 MP3 文件每个帧的 Bitrate 是固定不变的，所以每个帧都有相同的长度，称为 CBR，还有小部分 MP3 文件的 Bitrate 是可变，称之为 VBR，它是 XING 公司推出的算法。

MAIN_DATA 保存的是经过压缩算法压缩后得到的数据，为得到大的压缩率会损失部分源声音，属于失真压缩。

42.7.2 MP3 解码库

MP3 文件是经过压缩算法压缩而存在的，为得到 PCM 信号，需要对 MP3 文件进行解码，解码过程大致为：比特流分析、霍夫曼编码、逆量化处理、立体声处理、频谱重排列、抗锯齿处理、IMDCT 变换、子带合成、PCM 输出。整个过程涉及很多算法计算，要自己编程实现不是一件现实的事情，还好有很多公司经过长期努力实现了解码库编程。

现在合适在小型嵌入式控制器移植运行的有两个版本的开源 MP3 解码库，分别为 Libmad 解码库和 Helix 解码库，Libmad 是一个高精度 MPEG 音频解码库，支持 MPEG-1、MPEG-2 以及 MPEG-2.5 标准，它可以提供 24bitPCM 输出，完全是定点计算，更多信息可参考网站：<http://www.underbit.com/>。

Helix 解码库支持浮点和定点计算实现，将该算法移植到 STM32 控制器运行使用定点计算实现，它支持 MPEG-1、MPEG-2 以及 MPEG-2.5 标准的 Layer3 解码。Helix 解码库支持可变位速率、恒定位速率，以及立体声和单声道音频格式。更多信息可参考网站：<https://datatype.helixcommunity.org/Mp3dec>。

因为 Helix 解码库需要占用的资源比 Libmad 解码库更少，特别是 RAM 空间的使用，这对 STM32 控制器来说是比较重要的，所以在实验工程中我们选择 Helix 解码库实现 MP3 文件解码。这两个解码库都是一帧为解码单位的，一次解码一帧，这在应用解码库时是需要着重注意的。

Helix 解码库涉及算法计算，整个界面过程复杂，有兴趣可以深入探究，这里我们着重讲解 Helix 移植和使用方法。

Helix 网站有提供解码库代码，经过整理，移植 Helix 解码库需要用到的文件如图 42-13。有优化解码速度，部分解码过程使用汇编实现。

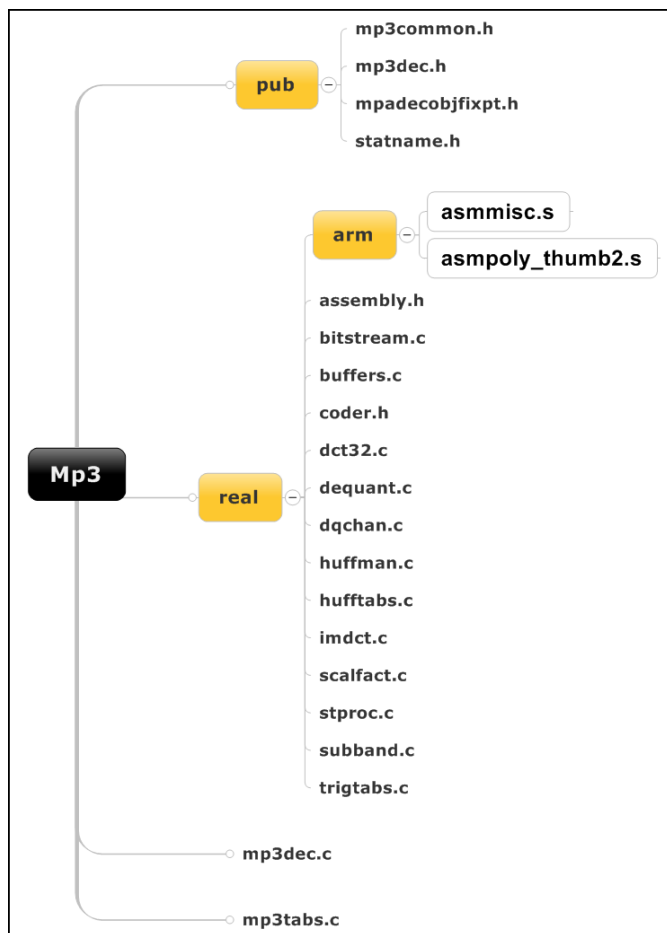


图 42-13 Helix 解码库文件结构

42.7.3 Helix 解码库移植

在“录音与回放实验”已经实现了 WM8978 驱动代码，现在我们可以移植 Helix 解码库工程中，实现 MP3 文件解码，将解码输出的 PCM 数据通过 I2S 接口发送到 WM8978 芯片实现音乐播放。

我们在“录音与回放实验”工程文件基础上移植 Helix 解码，首先将需要用到的文件添加到工程中，如图 42-14。MP3 文件夹下文件是 Helix 解码库源码，工程移植中是不需要修改文件夹下代码的，我们只需直接调用相关解码函数即可。建议自己移植时直接使用例程中 mp3 文件夹内文件。我们是在 mp3Player.c 文件中调用 Helix 解码库相关函数实现 MP3 文件解码的，该文件是我们自己创建的。

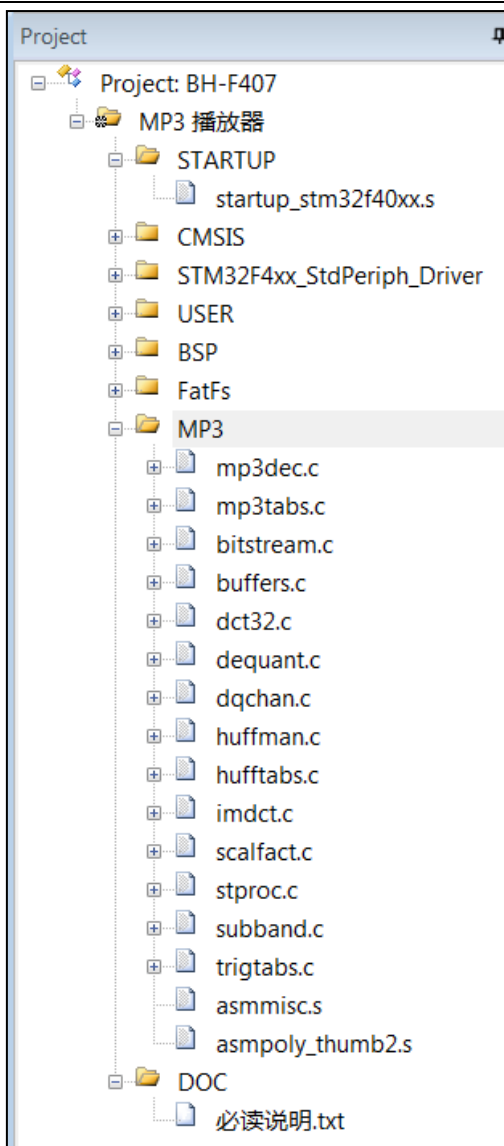


图 42-14 添加 Helix 解码库文件到工程

接下来还需要在工程选项中添加 Helix 解码库的文件夹路径，编译器可以寻找到相关头文件，见图 42-15。

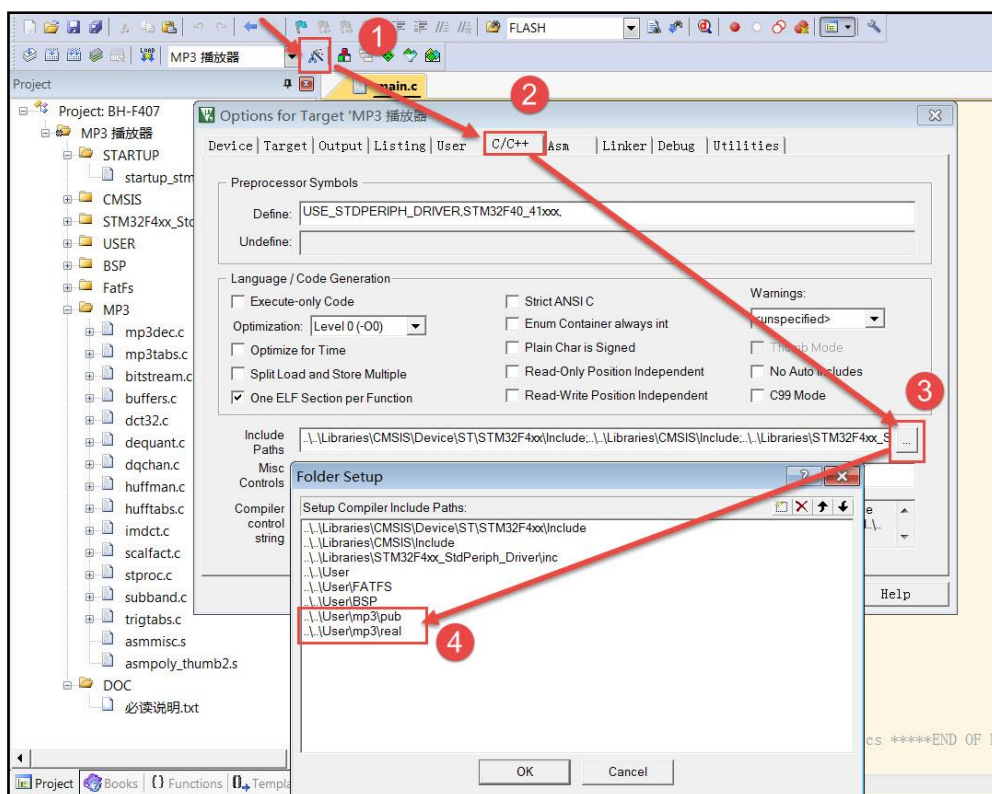


图 42-15 添加 Helix 解码库文件夹路径

42.7.4 MP3 播放器功能实现

“录音与回放实验”中的回放功能实际上就是从 SD 卡内读取 WAV 格式文件数据，然后提取里边音频数据通过 I2S 传输到 WM8978 芯片内实现声音播放。MP3 播放器的功能也是类似的，只不过现在音频数据提取方法不同，MP3 需要先经过解码库解码后才可得到“可直接”播放的音频数据。由此可以看到，MP3 播放器只是添加了 MP3 解码库实现代码，在硬件设计上并没有任何改变，即这里直接使用“录音与回放实验”中硬件设计即可。

实验工程代码中创建 mp3Player.c 和 mp3Player.h 两个文件存放 MP3 播放器实现代码。Helix 解码库是用来解码 MP3 数据帧，一次解码一帧，它是不能用来检索 ID3V1 和 ID3V2 标签的，如果需要获取歌名、作者等信息需要自己编程实现。解码过程可能用到的 Helix 解码库函数有：

- ❑ MP3InitDecoder：初始化解码器函数
- ❑ MP3FreeDecoder：关闭解码器函数
- ❑ MP3FindSyncWord：寻找帧同步函数
- ❑ MP3Decode：解码 MP3 帧函数
- ❑ MP3GetLastFrameInfo：获取帧信息函数

MP3InitDecoder 函数初始化解码器，它会申请分配一个存储空间用于存放解码器状态的一个数据结构并将其初始化，该数据结构由 MP3DecInfo 结构体定义，它封装了解码器内部运算数据信息。MP3InitDecoder 函数会返回指向该数据结构的指针。

MP3FreeDecoder 函数用于关闭解码器，释放由 MP3InitDecoder 函数申请的存储空间，所以一个 MP3InitDecoder 函数都需要有一个 MP3FreeDecoder 函数与之对应。它有一个形参，一般由 MP3InitDecoder 函数的返回指针赋值。

MP3FindSyncWord 函数用于寻址数据帧同步信息，实际上就是寻址数据帧开始的 11bit 都为“1”的同步信息。它有两个形参，第一个为源数据缓冲区指针，第二个为缓冲区大小，它会返回一个 int 类型变量，用于指示同步字较缓冲区起始地址的偏移量，如果在缓冲区中找不到同步字，则直接返回-1。

MP3Decode 函数用于解码数据帧，它有五个形参，第一个为解码器数据结构指针，一般由 MP3InitDecoder 函数返回值赋值；第二个参数为指向解码源数据缓冲区开始地址的一个指针，注意这里是地址的指针，即是指针的指针；第三个参数是一个指向存放解码源数据缓冲区有效数据量的变量指针；第四个参数是解码后输出 PCM 数据的指针，一般由我们定义的缓冲区地址赋值，对于双声道输出数据缓冲区以 LRLRLR...顺序排列；第五个参数是数据格式选择，一般设置为 0 表示标准的 MPEG 格式。函数还有一个返回值，用于返回解码错误，返回 ERR_MP3_NONE 说明解码正常。

MP3GetLastFrameInfo 函数用于获取数据帧信息，它有两个形参，第一个为解码器数据结构指针，一般由 MP3InitDecoder 函数返回值赋值；第二个参数为数据帧信息结构体指针，该结构体定义见代码清单 42-25。

代码清单 42-25 MP3 数据帧信息结构体

```
1 typedef struct _MP3FrameInfo {
2     int bitrate;           //位率
3     int nChans;           //声道数
4     int samprate;         //采样率
5     int bitsPerSample;    //采样位数
6     int outputSamps;      //PCM 数据数
7     int layer;            //层级
8     int version;          //版本
9 } MP3FrameInfo;
```

该结构体成员包括了该数据帧的位率、声道、采样频率等信息，它实际上是从数据帧的帧头信息中提取的。

MP3 播放器实现

代码清单 42-26 mp3PlayerDemo 函数

```
1 void mp3PlayerDemo(const char *mp3file)
2 {
3     uint8_t *read_ptr=inputbuf;
4     uint32_t frames=0;
5     int err=0, i=0, outputSamps=0;
6     int read_offset = 0;      /* 读偏移指针 */
7     int bytes_left = 0;      /* 剩余字节数 */
8
9     mp3player.ucFreq=I2S_AudioFreq_Default;
10    mp3player.ucStatus=STA_IDLE;
11    mp3player.ucVolume=40;
12
13    result=f_open(&file,mp3file,FA_READ);
14    if (result!=FR_OK) {
15        printf("Open mp3file :%s fail!!!->%d\r\n",mp3file,result);
16        result = f_close (&file);
17        return; /* 停止播放 */
18    }
```

```
19     printf("当前播放文件 -> %s\n",mp3file);
20
21     //初始化 MP3 解码器
22     Mp3Decoder = MP3InitDecoder();
23     if (Mp3Decoder==0) {
24         printf("初始化 helix 解码库设备\n");
25         return; /* 停止播放 */
26     }
27     printf("初始化中...\n");
28
29     Delay_ms(10); /* 延迟一段时间, 等待 I2S 中断结束 */
30     wm8978_Reset(); /* 复位 WM8978 到复位状态 */
31
32     /* 配置 WM8978 芯片, 输入为 DAC, 输出为耳机 */
33     wm8978_CfgAudioPath(DAC_ON, EAR_LEFT_ON | EAR_RIGHT_ON);
34
35     /* 调节音量, 左右相同音量 */
36     wm8978_SetOUT1Volume(mp3player.ucVolume);
37
38     /* 配置 WM8978 音频接口为飞利浦标准 I2S 接口, 16bit */
39     wm8978_CfgAudioIF(I2S_Standard_Phillips, 16);
40
41     /* 初始化并配置 I2S */
42     I2S_Stop();
43     I2S_GPIO_Config();
44     I2Sx_Mode_Config(I2S_Standard_Phillips,I2S_DataFormat_16b, mp3player.ucFreq);
45
46     I2S_DMA_TX_Callback=MP3Player_I2S_DMA_TX_Callback;
47     I2Sx_TX_DMA_Init((uint16_t *)outbuffer[0],(uint16_t *)outbuffer[1], MP3BUFFER_SIZE);
48
49
50     bufflag=0;
51     Isread=0;
52
53     mp3player.ucStatus = STA_PLAYING; /* 放音状态 */
54     result=f_read(&file,inputbuf,INPUTBUF_SIZE,&bw);
55     if (result!=FR_OK) {
56         printf("读取%s 失败 -> %d\r\n",mp3file,result);
57         MP3FreeDecoder(Mp3Decoder);
58         return;
59     }
60     read_ptr=inputbuf;
61     bytes_left=bw;
62     /* 进入主程序循环体 */
63     while (mp3player.ucStatus == STA_PLAYING) {
64         //寻找帧同步, 返回第一个同步字的位置
65         read_offset = MP3FindSyncWord(read_ptr, bytes_left);
66         //没有找到同步字
67         if (read_offset < 0) {
68             result=f_read(&file,inputbuf,INPUTBUF_SIZE,&bw);
69             if (result!=FR_OK) {
70                 printf("读取%s 失败 -> %d\r\n",mp3file,result);
71                 break;
72             }
73             read_ptr=inputbuf;
74             bytes_left=bw;
75             continue;
76         }
77
78         read_ptr += read_offset; //偏移至同步字的位置
79         bytes_left -= read_offset; //同步字之后的数据大小
80         if (bytes_left < 1024) { //补充数据
81             /* 注意这个地方因为采用的是 DMA 读取, 所以一定要 4 字节对齐 */
82             i=(uint32_t) (bytes_left)&3; //判断多余的字节
```



```
83         if (i) i=4-i; //需要补充的字节
84         memcpy(inputbuf+i, read_ptr, bytes_left); //从对齐位置开始复制
85         read_ptr = inputbuf+i; //指向数据对齐位置
86         //补充数据
87         result=f_read(&file,inputbuf+bytes_left+i,
88                     INPUTBUF_SIZE-bytes_left-i,&bw);
89         bytes_left += bw; //有效数据流大小
90     }
91     //开始解码 参数: mp3 解码结构体、输入流指针、输入流大小、输出流指针、数据格式
92     err = MP3Decode(Mp3Decoder, &read_ptr, &bytes_left,
93                   outbuffer[bufflag], 0);
94     frames++;
95     //错误处理
96     if (err != ERR_MP3_NONE) {
97         switch (err) {
98             case ERR_MP3_INDATA_UNDERFLOW:
99                 printf("ERR_MP3_INDATA_UNDERFLOW\r\n");
100                 result = f_read(&file, inputbuf, INPUTBUF_SIZE, &bw);
101                 read_ptr = inputbuf;
102                 bytes_left = bw;
103                 break;
104             case ERR_MP3_MAINDATA_UNDERFLOW:
105                 /* do nothing - next call to decode will provide more mainData */
106                 printf("ERR_MP3_MAINDATA_UNDERFLOW\r\n");
107                 break;
108             default:
109                 printf("UNKNOWN ERROR:%d\r\n", err);
110                 // 跳过此帧
111                 if (bytes_left > 0) {
112                     bytes_left --;
113                     read_ptr ++;
114                 }
115                 break;
116         }
117         Isread=1;
118     } else { //解码无错误, 准备把数据输出到 PCM
119         MP3GetLastFrameInfo(Mp3Decoder, &Mp3FrameInfo); //获取解码信息
120         /* 输出到 DAC */
121         outputSamps = Mp3FrameInfo.outputSamps; //PCM 数据个数
122         if (outputSamps > 0) {
123             if (Mp3FrameInfo.nChans == 1) { //单声道
124                 //单声道数据需要复制一份到另一个声道
125                 for (i = outputSamps - 1; i >= 0; i--) {
126                     outbuffer[bufflag][i * 2] = outbuffer[bufflag][i];
127                     outbuffer[bufflag][i * 2 + 1] = outbuffer[bufflag][i];
128                 }
129                 outputSamps *= 2;
130             } //if (Mp3FrameInfo.nChans == 1) //单声道
131         } //if (outputSamps > 0)
132
133         /* 根据解码信息设置采样率 */
134         if (Mp3FrameInfo.samprate != mp3player.ucFreq) { //采样率
135             mp3player.ucFreq = Mp3FrameInfo.samprate;
136
137             printf(" \r\n Bitrate      %dKbps", Mp3FrameInfo.bitrate/1000);
138             printf(" \r\n Samprate      %dHz", mp3player.ucFreq);
139             printf(" \r\n BitsPerSample %db", Mp3FrameInfo.bitsPerSample);
140             printf(" \r\n nChans      %d", Mp3FrameInfo.nChans);
141             printf(" \r\n Layer      %d", Mp3FrameInfo.layer);
142             printf(" \r\n Version      %d", Mp3FrameInfo.version);
143             printf(" \r\n OutputSamps %d", Mp3FrameInfo.outputSamps);
144             printf("\r\n");
145             //I2S_AudioFreq_Default = 2, 正常的帧, 每次都要改速率
146             if (mp3player.ucFreq >= I2S_AudioFreq_Default) {
147                 //根据采样率修改 I2S 速率
```

```
148         I2Sx_Mode_Config(I2S_Standard_Phillips,I2S_DataFormat_16b,  
149                           mp3player.ucFreq);  
150         I2Sx_TX_DMA_Init((uint16_t *)outbuffer[0],  
151                           (uint16_t *)outbuffer[1],outputSamps);  
152     }  
153     I2S_Play_Start();  
154 }  
155 //else 解码正常  
156  
157 if (file.fptr==file.fsize) { //mp3 文件读取完成, 退出  
158     printf("END\r\n");  
159     break;  
160 }  
161  
162 while (Isread==0) {  
163 }  
164 Isread=0;  
165 }  
166 I2S_Stop();  
167 mp3player.ucStatus=STA_IDLE;  
168 MP3FreeDecoder(Mp3Decoder);  
169 f_close(&file);  
170 }
```

mp3PlayerDemo 函数是 MP3 播放器的实现函数, 篇幅很长, 需要我们仔细分析。它有一个形参, 用于指定待播放的 MP3 文件, 需要用 MP3 文件的绝对路径加全名称赋值。

read_ptr 是定义的一个指针变量, 它用于指示解码器源数据地址, 把它初始化为用来存放解码器源数据缓冲区(inputbuf 数组)的首地址。read_offset 和 bytes_left 主要用于 MP3FindSyncWord 函数, read_offset 用来指示帧同步相对解码器源数据缓冲区首地址的偏移量, bytes_left 用于指示解码器源数据缓冲区有效数据量。

mp3player 是一个 MP3_TYPE 结构体类型变量, 指示音量、状态和采样频率信息。

f_open 函数用于打开文件, 如果文件打开失败则直接退出播放。MP3InitDecoder 函数用于初始化 Helix 解码器, 分配解码器必须内存空间, 如果初始化解码器失败直接退出播放。

接下来配置 WM8978 芯片功能, 使能耳机输出, 设置音量, 使用 I2S Philips 标准和 16bit 数据长度。还要设置 I2S 外设工作环境, 同样是 I2S Philips 标准和 16bit 数据长度, 采样频率先使用 I2S_AudioFreq_Default, 在运行 MP3GetLastFrameInfo 函数获取数据帧的采样频率后需要再次修改。MP3Player_I2S_DMA_TX_Callback 是一个函数名, 该函数实现 DMA 双缓冲区标志位切换以及指示 DMA 传输完成, 它在 I2S 的 DMA 发送传输完成中断服务函数中调用。I2Sx_TX_DMA_Init 用于初始化 I2S 的 DMA 发送请求, 使用双缓冲区模式。bufflag 用于指示当前 DMA 传输的缓冲区号, Isread 用于指示 DMA 传输完成。

f_read 函数从 SD 卡读取 MP3 文件数据, 存放在 inputbuf 缓冲区中, bw 变量保存实际读取到的数据的字节数。如果读取数据失败则运行 MP3FreeDecoder 函数关闭解码器后退出播放器。

接下来是循环解码帧数据并播放。MP3FindSyncWord 用于选择帧同步信息, 如果在源数据缓冲区中找不到同步信息, read_offset 值为-1, 需要读取新的 MP3 文件数据, 重新寻找帧同步信息。一般 MP3 起始部分是 ID3V2 信息, 所以可能需要循环几次才能寻找到帧同步信息。如果找到帧同步信息说明找到数据帧, 接下来数据就是数据帧的帧头以及 MAIN_DATA。

有时找到了帧同步信息，但可能源数据缓冲区并没有包括整帧数据，这时需要把从帧同步信息开始的源数据，复制到源数据缓冲区起始地址上，再使用 `f_read` 函数读取新数据填满整个源数据缓冲区，保证源数据缓冲区保存有整帧源数据。

`MP3Decode` 函数开始对源数据缓冲区中帧数据进行解码，通过函数返回值可判断得到解码状态，如果发生解码错误则执行对应的代码。

在解码无错误时，就可以使用 `MP3GetLastFrameInfo` 函数获取帧信息，如果有数据输出并且是单声道需要把数据复制成双声道数据格式。如果采样频率与上一次有所不同则执行通过串口打印帧信息到串口调试助手，可能还需要调整 I2S 的工作环境，还调用 `I2S_Play_Start` 启动播放。因为 `mp3player.ucFreq` 缺省值为 `I2S_AudioFreq_Default`，一般都不会与 `Mp3FrameInfo.samprate` 相等，所以会至少进入 `if` 语句内，即会执行 `I2S_Play_Start` 函数。

如果文件读取已经到了文件末尾就退出循环，MP3 文件已经播放完整。循环中还需要等待 DMA 数据传输完成才进行下一帧的解码操作。

`mp3PlayerDemo` 函数最后停止 I2S，关闭解码器和 MP3 文件。

DMA 发送完成中断回调函数

代码清单 42-27 MP3Player_I2S_DMA_TX_Callback 函数

```
1 void MP3Player_I2S_DMA_TX_Callback(void)
2 {
3     if (I2Sx_TX_DMA_STREAM->CR&(1<<19)) { //当前使用 Memory1 数据
4         bufflag=0;                          //可以将数据读取到缓冲区 0
5     } else {                                //当前使用 Memory0 数据
6         bufflag=1;                          //可以将数据读取到缓冲区 1
7     }
8     Isread=1;                              // DMA 传输完成标志
9 }
```

`MP3Player_I2S_DMA_TX_Callback` 函数用于在 DMA 发送完成后切换缓冲区。DMA 数据流 `x` 配置寄存器的 `CT` 位用于指示当前目标缓冲区，如果为 1，当前目标缓冲区为存储器 1；如果为 0，则为存储器 0。

主函数

代码清单 42-28 main 函数

```
1 int main(void)
2 {
3     FRESULT result;
4
5     /* NVIC 中断优先级组选择 */
6     NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2);
7
8     /* 关闭 BL_8782wifi 使能 */
9     BL8782_PDN_INIT();
10
11     /* 初始化调试串口，一般为串口 1 */
12     Debug_USART_Config();
13
14     /* 挂载 SD 卡文件系统 */
15     result = f_mount(&fs,"0:",1); //挂载文件系统
16     if (result!=FR_OK) {
17         printf("\n SD 卡文件系统挂载失败\n");
18         while (1);
19     }
```

```
20
21  /* 初始化系统滴答定时器 */
22  SysTick_Init();
23  printf("MP3 播放器\n");
24
25  /* 检测 WM8978 芯片, 此函数会自动配置 CPU 的 GPIO */
26  if (wm8978_Init()==0) {
27      printf("检测不到 WM8978 芯片!!!\n");
28      while (1); /* 停机 */
29  }
30  printf("初始化 WM8978 成功\n");
31
32  while (1) {
33      mp3PlayerDemo("0:/mp3/张国荣-玻璃之情.mp3");
34      mp3PlayerDemo("0:/mp3/张国荣-全赖有你.mp3");
35  }
36 }
```

main 函数主要完成各个外设的初始化, 包括初始化禁用 WIFI 模块、调试串口初始化、SD 卡文件系统挂载还有系统滴答定时器初始化。

wm8978_Init 初始化 I2C 接口用于控制 WM8978 芯片, 并复位 WM8978 芯片, 如果初始化成功则进入无限循环, 执行 MP3 播放器实现函数 mp3PlayerDemo, 它有一个形参, 用于指定播放文件。

另外, 为使程序正常运行还需要适当增加控制器的栈空间, 见代码清单 42-29, Helix 解码过程需要用到较多局部变量, 需要调整栈空间, 防止栈空间溢出。

代码清单 42-29 栈空间大小调整

```
1 Stack_Size      EQU      0x00001000
2
3 AREA            STACK, NOINIT, READWRITE, ALIGN=3
4 Stack_Mem       SPACE    Stack_Size
```

42.7.5 下载验证

将工程文件夹中的“音频文件放在 SD 卡根目录下”文件夹的内容拷贝到 Micro SD 卡根目录中, 把 Micro SD 卡插入到开发板右侧的卡槽内, 使用 USB 线连接开发板上的

“USB TO UART”接口到电脑, 电脑端配置好串口调试助手参数, 在开发板的上边沿的耳机插座(左边那个)插入耳机。编译实验程序并下载到开发板上, 程序运行后在串口调试助手可接收到开发板发过来的提示信息, 如果没有提示错误信息则直接在耳机可听到音乐, 播放完后自动切换下一首, 如此循环。

实验主要展示 MP3 解码库移植过程和实现简单 MP3 文件播放, 跟实际意义上的 MP3 播放器在功能上还有待完善, 比如快进快退功能、声音调节、音效调节等等。